ANATRA 1.1.1 User Guide

Kento Kasahara@The University of Osaka

Jun 14, 2026

CONTENTS:

CHAPTER

ONE

WHAT IS ANATRA?

ANATRA (*Ana*lyze *Tra*jectories) is a collection of Tcl/Fortran90 programs for analyzing trajectories obtained from Molecular Dynamics (MD) simulations.

We often need to analyze specific atoms, molecules, or amino acid residues of proteins in a system. However, writing a general-purpose program to extract such specific parts is not an easy task. This is because it is necessary to extract atom groups that satisfy multiple complex conditions simultaneously, such as:

> "residues numbered from 11 to 50, with residue name ALA, and only heavy atoms."

Creating a program that can interpret and extract such sets defined by intersections and unions of multiple conditions can be quite difficult.

On the other hand, **VMD** (*Visual Molecular Dynamics*), a tool widely used by MD users around the world, offers an excellent feature called **Atomselection** for selecting specific parts of a system. For instance, the above condition can be expressed in VMD′s Atomselection syntax as:

```
resid 11 to 50 and resname ALA and noh
```

which is easy to understand intuitively.

ANATRA adopts a design that leverages VMD′s Atomselection feature. As a result, users can use exactly the same syntax as in Atomselection for selecting specific parts of the system, eliminating the need to learn a new selection language—especially convenient for existing VMD users.

ANATRA consists of numerous Tcl/Fortran programs tailored to different types of analyses. To manage these programs in a unified way, a command-line interface named `"anatra"` is provided. Users can perform various analyses by executing:

```
$ anatra analysis_mode -option1 -option2 ...
```

In this workflow, the corresponding Tcl script is executed on VMD according to the specified analysis mode, utilizing Atomselection to identify the region of interest. The selected information is then passed as input to a Fortran program for analysis. All of this is handled internally, and users do not need to be aware of the intermediate steps—everything runs in a streamlined manner. At the same time, since the Fortran programs themselves do not implement Atomselection directly, the source code remains simpler and easier to maintain. This also allows users to create new analysis programs with ease by building upon the libraries provided by **ANATRA**.

- **Project Leader/Main Developer**
  - Kento Kasahara (Graduate School of Engineering Science, Univ of Osaka)
- **Contributor**
  - Ren Masayama (Graduate School of Engineering Science, Univ. of Osaka)
  - Kazuya Okita (Graduate School of Engineering Science, Univ. of Osaka)
  - Yuya Matsubara (Graduate School of Engineering Science, Univ. of Osaka)
  - Yusei Maruyama (Graduate School of Engineering Science, Univ. of Osaka)
  - Ryo Okabe (Graduate School of Engineering Science, Univ. of Osaka)
  - Yuuki Yamashita (Graduate School of Engineering Science, Univ. of Osaka)
  - Shuto Saeki (Graduate School of Engineering Science, Univ. of Osaka)
  - Nobuyuki Matubayasi (Graduate School of Engineering Science, Univ. of Osaka)

---

2. Redistributions in binary form must reproduce the above copyright notice, this list of conditions, and the following disclaimer in the documentation and/or materials provided with the distribution.
3. Neither the name of The HDF Group, the name of the University, nor the name of any Contributor may be used to endorse or promote products derived from this software without specific prior written permission from The HDF Group, the University, or the Contributor, respectively.

DISCLAIMER: THIS SOFTWARE IS PROVIDED BY THE COPYRIGHT HOLDERS AND CONTRIBUTORS "AS IS" AND ANY EXPRESS OR IMPLIED WARRANTIES, INCLUDING, BUT NOT LIMITED TO, THE IMPLIED WARRANTIES OF MERCHANTABILITY AND FITNESS FOR A PARTICULAR PURPOSE ARE DISCLAIMED. IN NO EVENT SHALL THE COPYRIGHT HOLDER OR CONTRIBUTORS BE LIABLE FOR ANY DIRECT, INDIRECT, INCIDENTAL, SPECIAL, EXEMPLARY, OR CONSEQUENTIAL DAMAGES (INCLUDING, BUT NOT LIMITED TO, PROCUREMENT OF SUBSTITUTE GOODS OR SERVICES; LOSS OF USE, DATA, OR PROFITS; OR BUSINESS INTERRUPTION) HOWEVER CAUSED AND ON ANY THEORY OF LIABILITY, WHETHER IN CONTRACT, STRICT LIABILITY, OR TORT (INCLUDING NEGLIGENCE OR OTHERWISE) ARISING IN ANY WAY OUT OF THE USE OF THIS SOFTWARE, EVEN IF ADVISED OF THE POSSIBILITY OF SUCH DAMAGE.

For further details, please refer to the full license text available at https://opensource.org/licenses/bsd-3-clause

You are under no obligation whatsoever to provide any bug fixes, patches, or upgrades to the features, functionality or performance of the source code ("Enhancements") to anyone; however, if you choose to make your Enhancements available either publicly, or directly to The HDF Group, without imposing a separate written license agreement for such Enhancements, then you hereby grant the following license: a non-exclusive, royalty-free perpetual license to install, use, modify, prepare derivative works, incorporate into other computer software, distribute, and sublicense such enhancements or derivative works thereof, in binary and source code form.

- **NetCDF**
  Copyright 2025 Unidata

  Redistribution and use in source and binary forms, with or without modification, are permitted provided that the following conditions are met:

  1. Redistributions of source code must retain the above copyright notice, this list of conditions and the following disclaimer.
  2. Redistributions in binary form must reproduce the above copyright notice, this list of conditions and the following disclaimer in the documentation and/or other materials provided with the distribution.
  3. Neither the name of the copyright holder nor the names of its contributors may be used to endorse or promote products derived from this software without specific prior written permission.

  THIS SOFTWARE IS PROVIDED BY THE COPYRIGHT HOLDERS AND CONTRIBUTORS "AS IS" AND ANY EXPRESS OR IMPLIED WARRANTIES, INCLUDING, BUT NOT LIMITED TO, THE IMPLIED WARRANTIES OF MERCHANTABILITY AND FITNESS FOR A PARTICULAR PURPOSE ARE DISCLAIMED. IN NO EVENT SHALL THE COPYRIGHT HOLDER OR CONTRIBUTORS BE LIABLE FOR ANY DIRECT, INDIRECT, INCIDENTAL, SPECIAL, EXEMPLARY, OR CONSEQUENTIAL DAMAGES (INCLUDING, BUT NOT LIMITED TO, PROCUREMENT OF SUBSTITUTE GOODS OR SERVICES; LOSS OF USE, DATA, OR PROFITS; OR BUSINESS INTERRUPTION) HOWEVER CAUSED AND ON ANY THEORY OF LIABILITY, WHETHER IN CONTRACT,

STRICT LIABILITY, OR TORT (INCLUDING NEGLIGENCE OR OTHERWISE) ARISING IN ANY WAY OUT OF THE USE OF THIS SOFTWARE, EVEN IF ADVISED OF THE POSSIBILITY OF SUCH DAMAGE.

- **xdrfile-1.1.4**
  Copyright © 2009-2014, Erik Lindahl & David van der Spoel
  All rights reserved.

  Redistribution and use in source and binary forms, with or without modification, are permitted provided that the following conditions are met:

  1. Redistributions of source code must retain the above copyright notice, this list of conditions and the following disclaimer.
  2. Redistributions in binary form must reproduce the above copyright notice, this list of conditions and the following disclaimer in the documentation and/or other materials provided with the distribution.

  THIS SOFTWARE IS PROVIDED BY THE COPYRIGHT HOLDERS AND CONTRIBUTORS "AS IS" AND ANY EXPRESS OR IMPLIED WARRANTIES, INCLUDING, BUT NOT LIMITED TO, THE IMPLIED WARRANTIES OF MERCHANTABILITY AND FITNESS FOR A PARTICULAR PURPOSE ARE DISCLAIMED. IN NO EVENT SHALL THE COPYRIGHT HOLDER OR CONTRIBUTORS BE LIABLE FOR ANY DIRECT, INDIRECT, INCIDENTAL, SPECIAL, EXEMPLARY, OR CONSEQUENTIAL DAMAGES (INCLUDING, BUT NOT LIMITED TO, PROCUREMENT OF SUBSTITUTE GOODS OR SERVICES; LOSS OF USE, DATA, OR PROFITS; OR BUSINESS INTERRUPTION) HOWEVER CAUSED AND ON ANY THEORY OF LIABILITY, WHETHER IN CONTRACT, STRICT LIABILITY, OR TORT (INCLUDING NEGLIGENCE OR OTHERWISE) ARISING IN ANY WAY OUT OF THE USE OF THIS SOFTWARE, EVEN IF ADVISED OF THE POSSIBILITY OF SUCH DAMAGE.

  https://ftp.gromacs.org/pub/contrib/

- **xdf.F90**
  This routine was originally developed by Wes Barnett and distributed under the GNU General Public License as published by the Free Software Foundation; either version 2 of the License, or (at your option) any later version.
  https://github.com/wesbarnett/libgmxfort
  Modified version of the routine was developed by Kai-Min Tu.
  https://github.com/kmtu/xdrfort

- **mt19937.f**
  This routine was developed by Makoto Matsumoto and Takuji Nishimura and was distributed under the GNU Library General Public License as published by the Free Software Foundation; either version 2 of the License, or (at your option) any later version.
  https://www.math.sci.hiroshima-u.ac.jp/m-mat/MT/VERSIONS/FORTRAN/fortran.html

- **FFTE**
  Copyright © 2000-2003 Daisuke Takahashi

  You may use, copy, modify this code for any purpose (include commercial use) and without fee. You may distribute this ORIGINAL package.

  https://www2.ccs.tsukuba.ac.jp/SC/sc2003/Software/FFTE_Package-3.0/doc/index.html

- **IEPDYN (Integral-Equation formalism of Population DYNamics)**

  https://github.com/kenkasa/iepdyn

CHAPTER

TWO

INSTALLATION OF ANATRA

This chapter describes how to install ANATRA.

2.1 Requirements

- VMD (> 1.7.3)
- Intel compiler (Intel OneAPI) or GCC (> 8.5.0)
- bash

ANATRA assumes that `bash` is used as the login shell.[1]

VMD can be downloaded free of charge from

```
https://www.ks.uiuc.edu/Development/Download/download.cgi?PackageName=VMD
```

although account registration is required.[2]

2.2 Installing from source

The latest ANATRA source code is managed on GitHub.

```
https://github.com/kenkasa/anatra
```

First, clone the repository. Assuming that ANATRA will be installed under `/home/USER/software`:

[1] As described in the next section, users of other shells such as `zsh` can also use ANATRA by adding the appropriate settings to their own rc files (e.g., `~/.zshrc`).

[2] A brief installation guide for VMD is given here. After extracting the downloaded `vmd-XXX.tar.gz` archive, move to the generated `vmd-XXX` directory. Open the `configure` file with a text editor and set `install_bin_dir=` near the beginning of the file to the directory where you want to install VMD. On Linux, run `./configure LINUXAMD64`, move to the `src` directory, and execute `make install`. The `vmd` executable will be installed into the directory specified by `install_bin_dir=`. Add this directory to your `PATH` to complete the installation.

```
$ cd /home/USER/software
$ git clone https://github.com/kenkasa/anatra.git
$ ls
anatra

$ cd anatra
$ ls
README.md
install.sh
bin/
tcl/
f90/
docs/
utility/
```

Installation is started by executing `install.sh`. If you have installed ANATRA previously, check `~/.bashrc` or `~/.zshrc` and remove the following three lines before installation:

```
export ANATRA_PATH=...
export PATH=$ANATRA_PATH/bin:$PATH
export LD_LIBRARY_PATH=$ANATRA_PATH/f90/lib/external/netcdf/netcdf/lib:$LD_LIBRARY_PATH
```

If `$ANATRA_PATH` already points to the same ANATRA directory that you are installing, these lines may be left unchanged. If you are unsure, simply remove them.

The available command-line options for `install.sh` are listed below.

Option	Value	Description
`--compiler=<`	`gcc`, `intel`	Specifies the compiler to be used.
`--install-hd`	—	Installs the HDF5 library.
`--skip-insta`	—	Skips the installation of external libraries.
`--with-openb`	—	Uses OpenBLAS instead of Intel MKL.The environment variable `OPENBLAS_ROOT` must be defined in advance to specify the OpenBLAS installation directory.
`--with-ffte`	—	Uses the FFTE library instead of the FFT implementation provided by Intel MKL.

The compiler can be specified with a command-line option:

```
$ ./install.sh --compiler=<gcc or intel>
```

The default is `intel`. If the Intel compiler is available, its use is recommended.

For Intel OneAPI:

```
$ ./install.sh --compiler=intel
```

Because ANATRA supports NetCDF trajectories, the HDF5 library is required. Many Linux distributions already provide HDF5. If HDF5 is not installed or you are unsure, add the `--install-hdf5` option to build the bundled HDF5 package:

```
$ ./install.sh --compiler=<gcc or intel> --install-hdf5
```

During installation, build logs are displayed. If the following message appears near the end of the build process, all Fortran analysis programs have been compiled successfully:

```
Installation of ANATRA fortran programs have been
succesfully finished!!
```

The installation is complete if the `ANATRA_PATH` environment variable is defined and the `anatra` command is available in your `PATH`:

```
$ echo $ANATRA_PATH
/home/USER/software/anatra

$ which anatra
/home/USER/software/anatra/bin/anatra
```

When rebuilding ANATRA after modifying the Fortran source code, executing `install.sh` with the standard options rebuilds all external libraries from scratch, which can be time-consuming. To avoid this, the `--skip-install-lib` option can be added to skip rebuilding the external libraries. However, if the compiler has been changed since the previous build, all external libraries must also be rebuilt.

Depending on the environment of the machine where ANATRA is installed, the installation may fail because some required libraries are missing. If this happens, please refer to the instructions below.

2.2.1 Building the HDF5 library

ANATRA requires the HDF5 library because it supports NetCDF-format trajectories.

Many Linux distributions provide HDF5 as a pre-installed system library. If HDF5 is not installed, or if you are unsure whether it is available, you can install the bundled HDF5 library by adding the `--install-hdf5` option:

```
$ ./install.sh --compiler=<gcc or intel> --install-hdf5
```

2.2.2 Using OpenBLAS

By default, `install.sh` compiles ANATRA using the Intel Math Kernel Library (MKL) included in Intel OneAPI. However, if Intel OneAPI is not installed, OpenBLAS can be used as an alternative.

OpenBLAS can be downloaded from GitHub:

```
$ git clone https://github.com/OpenMathLib/OpenBLAS.git
```

This command creates a directory named `OpenBLAS` containing the source code. Enter this directory and execute `make` followed by `make install`.

The following example installs the library under the `OpenBLAS` directory:

```
$ cd OpenBLAS
$ make
$ make PREFIX=/path/to/OpenBLAS install
```

After installation, define the global environment variable `OPENBLAS_ROOT` in `~/.bashrc` or `~/.zshrc` as follows:

```
export OPENBLAS_ROOT=/path/to/OpenBLAS
```

Then, run ANATRA's `install.sh` script with the `--with-openblas` option:

```
$ ./install.sh --compiler=gcc --with-openblas
```

When OpenBLAS is selected as the mathematical library, ANATRA automatically uses `fftе` as the Fast Fourier Transform (FFT) library.

CHAPTER

THREE

USAGE OF ANATRA

3.1 Introduction

anatra is a post-processing analysis suite that combines the use of VMD and Fortran programs. This chapter provides a detailed guide on how to use **anatra**. As described previously, the standard workflow in **anatra**-based analysis proceeds in two stages. First, atom or residue selection and preprocessing are carried out using VMD, resulting in intermediate files. These files are subsequently analyzed via Fortran executables.

A list of available analysis modes can be obtained by executing the following command:

```
$ anatra -h
Available analysis in ANATRA
trjconv             (trj)   : Convert trajectory
distance            (dis)   : Distance Analysis
center_of_mass      (com)   : Center-of-Mass Analysis
radial_distr        (rdf)   : Radial-Distribution Analysis
spatial_distr       (sdf)   : Spatial-Distribution Analysis
rmsd                (rmsd)  : Root-Mean-Square-Deviation Analysis
interaction_energy  (ene)   : Interaction Energy Analysis
lipid_order         (scd)   : Lipid S_CD order-parameter Analysis
area_per_lipid      (apl)   : Area Per Lipid Analysis
z_profile           (zprof) : Z-profile Analysis
z_orient            (zori)  : Orientation Analysis along z
rotation            (rot)   : Rotation Analysis
cluster_size        (cls)   : Cluster-size Analysis
free_volume         (fvol)  : Free-Volume Analysis
species_info        (spec)  : Get Species information
gauqm_prepper       (gau)   : Gaussian QM input prepper
```

For example, the `trjconv` analysis mode may be invoked as follows:

```
$ anatra trjconv -option1 ...
```

Abbreviated mode names, as shown in parentheses (e.g., `trj` for `trjconv`), can also be used:

```
$ anatra trj -option1 ...
```

To display available options and example usage for each mode, one can execute:

```
$ anatra <analysis_mode> -h
```

This help message is generally sufficient for basic usage. However, for more advanced configurations or clarification, this manual should be consulted.
An example help output for `trjconv` is shown below (VMD-related output is omitted for brevity):

```
$ anatra trjconv -h
============================================================

                    Trajectory Convert

============================================================
Usage:
anatra trjconv                                                    \
  -stype      <structure file type>                               \
  -sfile      <structure file name>                               \
  -tintype    <input trajectory file type>                        \
  -tin        <input trajectory file name>                        \
  -totype     <output trajectory file type>                       \
  -to         <output trajectory file name>                       \
  -beg        <first frame to be read> (default: 1)               \
  -end        <last frame to be read>                             \
              (default: 0, corresponding to the final frame)      \
  -selX       <X-th VMD selection> (X = 0,1,2...)                 \
  -fit        <whether to apply fitting (true or false)>          \
  -centering  <whether to apply centering (true or false)>        \
              (default: false)                                    \
  -wrap       <whether to apply wrapping (true or false)>         \
              (default: false)                                    \
  -wrapcenter <wrapping center: origin or com (default: fragment)>  \
  -wrapcomp   <wrapping unit: residue, segid, chain, fragment, or nomp>  \
              (default: fragment)                                 \
  -refpdb     <reference PDB file name>                           \
              (required if fit = true)                            \
  -outselid   <selection ID for output molecules>                 \
  -fitselid   <selection ID for fitting or centering>             \
  -refselid   <selection ID for reference>
```

(continues on next page)

(continued from previous page)

```
Remark:
o If both fit = true and wrap = true are specified, wrapcomp is automatically set to 'com'

Example (fitting):
anatra trjconv                          \
  -stype      parm7                     \
  -sfile      str.prmtop                \
  -tintype    dcd                       \
  -tin        inp.dcd                   \
  -totype     dcd                       \
  -to         out.dcd                   \
  -beg        1                         \
  -end        150                       \
  -sel0       not water                 \
  -sel1       resid 1 to 275 and name CA \
  -sel2       resid 1 to 275 and name CA \
  -fit        true                      \
  -centering  false                     \
  -wrap       true                      \
  -wrapcenter origin                    \
  -wrapcomp   fragment                  \
  -refpdb     ref.pdb                   \
  -outselid   0                         \
  -fitselid   1                         \
  -refselid   2
```

As illustrated above, **anatra** accepts a considerable number of options, making it well-suited for use within bash scripts. An example script is provided below:

```
#!/bin/bash

anatra trjconv            \
  -stype    parm7         \
  -sfile    str.prmtop    \
  -tintype  dcd           \
  -tin      INP.dcd       \
  -totype   dcd           \
  -to       OUT.dcd       \
  -sel0     resid 1 to 20 \
  -outselid 0
```

3.2 Common Options

While **anatra** offers various analysis modes, many of them share a common set of command-line options. A summary of these shared options is presented below, followed by detailed descriptions of each.

Option	Function	Example
`-stype`	Specifies the structure file type	`-stype parm7`
`-sfile`	Specifies the structure file name	`-sfile A.prmtop`
`-tintype`	Specifies the input trajectory file type	`-tintype dcd`
`-tin`	Specifies the input trajectory file name	`-tin in.dcd`
`-flist_traj`	Specifies a file listing multiple trajectory files	`-flist_traj list.txt`
`-totype`	Specifies the output trajectory file type	`-totype dcd`
`-to`	Specifies the output trajectory file name	`-to out.dcd`
`-sel`x	Specifies a selection using VMD syntax	`-sel0 resid 1 to 20`
`-mode or -mode`x	Specifies how to group the selected atoms	`-mode residue`
`-fhead`	Header for output file names	`-fhead out`
`-prep_only`	If true, only preprocessing is performed	`-prep_only true`

- **-stype <structure file type>**
  - Default: N/A
  - Example: `-stype parm7`

  Specifies the structure file type. Structure files contain information such as atom types and possibly bonding details. Since many trajectory formats (e.g., `dcd` or `xtc`) store only atomic coordinates, a structure file is required for analysis. Most of the structure file types supported by VMD can be specified. Representative examples include:
  - **pdb**: PDB file
  - **gro**: GRO file
  - **psf**: CHARMM PSF file
  - **parm7**: AMBER parameter file (usually with extension `.prmtop`)

  Among these, `psf` (for CHARMM) and `parm7` (for AMBER) provide the most complete information and are recommended when available.

- **-sfile <structure file>**
  - Default: N/A
  - Example: `-sfile complex.prmtop`

Specifies the name of the structure file.

- **-tintype <input trajectory file type>**
  - Default: `dcd`
  - Example: `-tintype dcd`

  Specifies the type of the input trajectory file. Currently, with the exception of `trjconv`, supported formats are limited to `dcd`, `xtc`, and `netcdf`. `trjconv` supports all trajectory formats recognized by VMD.

- **-tin <input trajectory file>**
  - Default: N/A
  - Example: `-tin INP.dcd`

  Specifies the filename of the input trajectory. When using multiple files, list them as: `-tin run1.dcd run2.dcd`. Wildcards (e.g., `run*.dcd`) are also supported. Alternatively, you can use the `-flist_traj` option described below.

- **-flist_traj <text file>**
  - Default: N/A
  - Example: `-flist_traj trajlist.txt`

  Specifies a text file listing the input trajectories. If both `-tin` and `-flist_traj` are given, `-flist_traj` takes precedence. The file should contain one trajectory filename per line, as shown below:

```
traj1.dcd
traj2.dcd
traj3.dcd
```

- **-totype <output trajectory file type>**
  - Default: `dcd`
  - Example: `-totype dcd`

  Specifies the type of the output trajectory file. As with input, all VMD-supported formats can be used in `trjconv`. Other modes support only `dcd` and `xtc`.

- **-to <output trajectory file>**
  - Default: N/A
  - Example: `-to OUT.dcd`

Specifies the name of the output trajectory file.

- **-sel**x **<selection>** ($x = 0, 1, \cdots$)
  - Default: N/A
  - Example: `-sel0 resid 1 to 20 and name CA`

  Specifies an atomic selection using VMD′s Atomselection syntax. The index x is referred to as the selection ID (*selid*). Most analysis modes perform calculations on the region specified by each `-sel$x$`. In some cases, you must explicitly assign which `selid` is used for which purpose (e.g., `-outselid` in `trjconv`).

- **-mode**x **<residue or whole or atom>** ($x = 0, 1, \cdots$)
  - Default: Mode-dependent
  - Example: `-mode0 residue`

  Specifies how to group the atoms selected by `-sel$x$` (see Fig. 3.1). Some analysis modes only require a single selection, using `-mode` to define grouping. Others require multiple selections, each needing a corresponding `-mode$x$`. See the mode-specific descriptions for details.
  - **residue**: Treat each residue in the selection as a separate molecule.
  - **whole**: Treat the entire selection as a single molecule.
  - **atom**: Treat each atom in the selection as an individual molecule.

  For example, if `-sel0 resname LIG` selects multiple ligand molecules and you wish to compute each center of mass, set `-mode residue`. Conversely, to treat a full protein selected via `-sel0 protein` as one object (e.g., for center of mass tracking), set `-mode whole`.

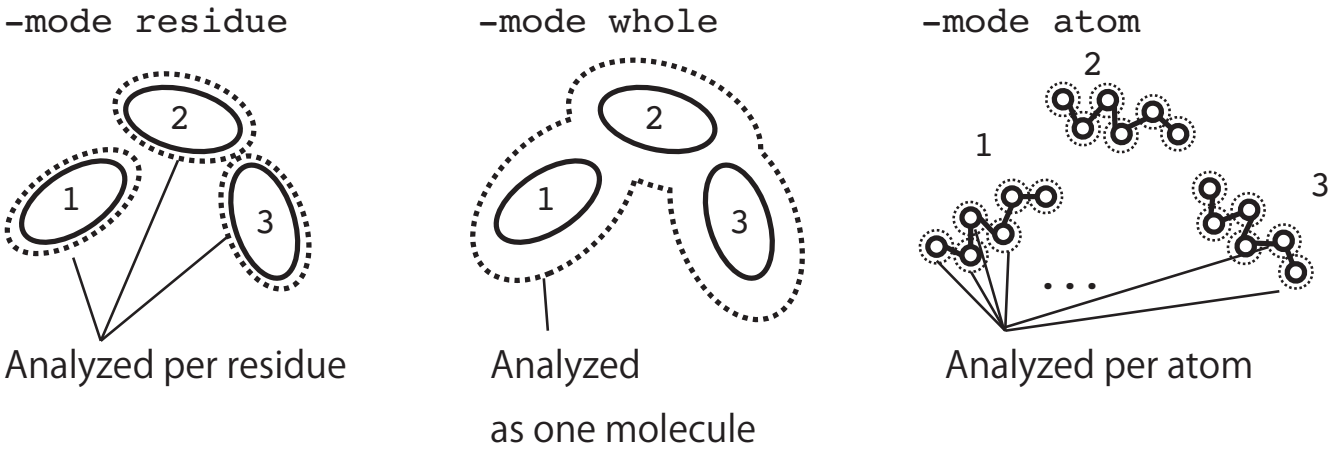

Fig. 3.1: Usage of the `-mode` option. In `residue`, the selected region is split by residue. In `whole`, the entire selection is treated as one molecule. In `atom`, each atom in the selection is treated as a separate molecule.

- **-fhead**
  - Default: Mode-dependent
  - Example: `-fhead out`

  Specifies the prefix for output filenames. For example, if set to `out`, output files like `out.XXX`, `out.YYY` will be generated.

- **-prep_only**
  - Default: `false`
  - Example: `-prep_only true`

  Specifies whether to stop after preprocessing (e.g., generating input parameter files) without running the analysis itself.

3.3 Trajectory Conversion (`trjconv`)

The `trjconv` mode performs various transformations on MD trajectories. It is typically used for the following purposes:

- To change the file format of a trajectory.
- To generate a trajectory containing only a specific region of interest
  (e.g., to remove water molecules from the trajectory).
- To apply periodic boundary wrapping (PBC wrapping) to an unwrapped trajectory.
- To fit the system such that a selected region aligns with a reference structure.
- To center the system such that a selected region is translated to the origin.

3.3.1 List of Options

Option	Remarks	Option	Remarks
`-stype`	Common	`-sfile`	Common
`-tintype`	Common	`-tin`	Common
`-totype`	Common	`-to`	Common
`-sel`x	Common	`-beg`	Mode specific
`-end`	Mode specific	`-fit`	Mode specific
`-centering`	Mode specific	`-wrap`	Mode specific
`-wrapcenter`	Mode specific	`-wrapcomp`	Mode specific
`-refpdb`	Mode specific	`-outselid`	Mode specific
`-fitselid`	Mode specific	`-refselid`	Mode specific

- **-beg <first frame to read>**
  - Default: `1`
  - Example: `-beg 10`

  Specifies the first frame to read from the trajectory. Note that frame indexing starts from 1. The example above starts reading from the 10th frame.

- **-end <last frame to read>**
  - Default: `0` (interpreted as the final frame)
  - Example: `-end 100`

  Specifies the last frame to read from the trajectory. Frame numbers are counted starting from 1. The example above reads up to the 100th frame.

- **-fit <true or false>**
  - Default: `false`
  - Example: `-fit true`
  - Note: If set to `true`, the options `-fitselid`, `-refpdb`, and `-refselid` must be specified.

  Enables or disables structural fitting. Fitting refers to the operation of translating and rotating the entire system so that a specific region matches a reference structure. When this option is used, the following must be specified:
  - `-fitselid`: selid of the fitting region in the trajectory
  - `-refpdb`: PDB file of the reference structure
  - `-refselid`: selid of the fitting region in the reference

- **-wrap <true or false>**
  - Default: `false`
  - Example: `-wrap true -wrapcomp fragment -wrapcenter origin`
  - Note: Use `-wrapcenter` to change the wrapping center from the origin.
    Use `-wrapcomp` to define the unit for wrapping.

  Specifies whether to apply periodic boundary wrapping (PBC) to the trajectory. By default, the wrapping center is the origin $(x, y, z) = (0, 0, 0)$, but it will be automatically changed to the center of mass of a region specified with `-wrapcenter`.

- **-centering <true or false>**
  - Default: `false`

  - Example: `-centering true -sel1 resid 1 to 200 -fitselid 1`
  - Note: Use `-fitselid` to specify the selid of the region to be centered.

  Specifies whether to perform centering of the system. Centering refers to translating the system so that the center of mass of a selected region is placed at the coordinate origin. The region to be centered is specified via `-fitselid`.

- **-refpdb <PDB file>**
  - Default: N/A
  - Example: `-refpdb A.pdb -sel2 resid 1 to 200 -refselid 2`
  - Note: Specify the selid of the fitting region in the reference using `-refselid`.

  Specifies the PDB file of the reference structure for fitting. This option is required if `-fit true` is specified. The fitting region within the reference is selected using `-refselid`.

- **-outselid <selid>**
  - Default: N/A
  - Example: `-sel0 resid 1 to 200 -outselid 0`

  Specifies the selid corresponding to the region to be written to the output trajectory.

- **-fitselid <selid>**
  - Default: N/A
  - Example: `-sel1 resid 1 to 200 -fitselid 1`

  Specifies the selid of the fitting region in the input trajectory.

- **-wrapcenter <origin or com>**
  - Default: `origin`
  - Example: `-wrapcenter origin`
  - Note: If `com` is specified, the selid must be provided using `-fitselid`.

  Specifies the center of the system used during wrapping.

  - **origin**: Use the coordinate origin as the center.
  - **com**: Use the center of mass of a selected region as the center.
    The selid of this region must be given with `-fitselid`.
    If fitting or centering is performed simultaneously, `com` is enforced.

- **-wrapcomp <fragment or residue or nocomp>**
  - Default: `fragment`
  - Example: `-wrapcomp fragment`

  Specifies the unit by which molecules are treated for PBC wrapping. For example, when atoms are partially outside the box, this determines whether to wrap individual atoms or entire molecular units.
  - **fragment**: Atoms connected through bonding paths are treated as a single unit.
  - **residue**: All atoms within a residue ID are treated as a single unit.
  - **nocomp**: Each atom is treated as an independent unit.

3.3.2 Examples of Analysis

Below are script examples using the minimum necessary options for common analyses. Options such as `-beg` and `-end` are omitted.

3.3.2.1 Removing Water from Trajectory

```
#!/bin/bash

anatra trjconv                          \
  -stype      parm7                     \
  -sfile      complex.prmtop            \
  -tintype    dcd                       \
  -tin        INP.dcd                   \
  -totype     dcd                       \
  -to         OUT.dcd                   \
  -sel0       not water                 \
  -outselid   0
```

3.3.2.2 PBC Wrapping

```
#!/bin/bash

anatra trjconv                          \
  -stype      parm7                     \
  -sfile      complex.prmtop            \
  -tintype    dcd                       \
```

(continues on next page)

(continued from previous page)

```
  -tin        INP.dcd                  \
  -totype     dcd                      \
  -to         OUT.dcd                  \
  -sel0       all                      \
  -wrap       true                     \
  -wrapcenter origin                   \
  -wrapcomp   fragment                 \
  -outselid   0
```

3.3.2.3 Fitting

This example assumes a system with one protein in aqueous solution (`resid 1 to 192`). Fitting is performed to align the Cα atoms in the trajectory to those in the reference structure:

```
anatra trjconv                         \
  -stype      parm7                    \
  -sfile      complex.prmtop           \
  -tintype    dcd                      \
  -tin        INP.dcd                  \
  -totype     dcd                      \
  -to         OUT.dcd                  \
  -refpdb     REF.pdb                  \
  -sel0       all                      \
  -sel1       resid 1 to 192 and name CA \
  -sel2       resid 1 to 192 and name CA \
  -fit        true                     \
  -outselid   0                        \
  -fitselid   1                        \
  -refselid   2
```

3.3.2.4 Centering

This example centers the protein′s center of mass at the origin:

```
anatra trjconv                         \
  -stype      parm7                    \
  -sfile      complex.prmtop           \
  -tintype    dcd                      \
  -tin        INP.dcd                  \
  -totype     dcd                      \
```

(continues on next page)

(continued from previous page)

```
  -to         OUT.dcd                   \
  -sel0       all                       \
  -sel1       resid 1 to 192            \
  -centering  true                      \
  -outselid   0                         \
  -fitselid   1
```

3.3.2.5 PBC Wrapping and Fitting

This example performs both fitting and PBC wrapping. The wrapping center is set to the center of mass of the Cα atoms:

```
anatra trjconv                          \
  -stype      parm7                     \
  -sfile      complex.prmtop            \
  -tintype    dcd                       \
  -tin        INP.dcd                   \
  -totype     dcd                       \
  -to         OUT.dcd                   \
  -refpdb     REF.pdb                   \
  -sel0       all                       \
  -sel1       resid 1 to 192 and name CA \
  -sel2       resid 1 to 192 and name CA \
  -fit        true                      \
  -wrap       true                      \
  -wrapcenter com                       \
  -wrapcomp   fragment                  \
  -outselid   0                         \
  -fitselid   1                         \
  -refselid   2
```

3.3.2.6 PBC Wrapping and Centering

This example sets the wrapping and centering region to a lipid membrane (`resid 1 to 200 and segid MEMB`):

```
anatra trjconv                          \
  -stype      psf                       \
  -sfile      complex.psf               \
  -tintype    dcd                       \
  -tin        INP.dcd                   \
  -totype     dcd                       \
```

(continues on next page)

(continued from previous page)

```
-to         OUT.dcd                        \
-sel0       all                            \
-sel1       resid 1 to 200 and segid MEMB  \
-centering  false                          \
-wrap       true                           \
-wrapcenter com                            \
-wrapcomp   fragment                       \
-outselid   0                              \
-fitselid   1
```

3.4 Distance Calculation (`distance`, `dis`)

The `distance` mode computes the distance between two selected regions. By default, it calculates the distance between the centers of mass of the selected molecules. It can also compute the minimum distance between atoms in two regions.

3.4.1 List of Options

Option	Remarks	Option	Remarks
`-stype`	Common	`-sfile`	Common
`-tintype`	Common	`-tin`	Common
`-flist_traj`	Common	`-fhead`	Common
`-sel0`	Common	`-sel1`	Common
`-mode0`	Common	`-mode1`	Common
`-pbc`	Mode specific	`-dt`	Mode specific
`-distance_type`†	Mode specific	`-mindist_type0`†	Mode specific
`-mindist_type1`†	Mode specific	`-prep_only`	Common

Options marked with † are not required for standard distance calculations and can be omitted in typical use cases.

- **-sel0 <selection>**
- **-sel1 <selection>**
  - Default: N/A
  - Example: `-sel0 resid 1 to 20 and noh -sel1 resname LIG`

  Specifies the two regions (selid = 0, 1) between which distances will be calculated.

- **-mode0 <residue or whole or atom>**
- **-mode1 <residue or whole or atom>**
  - Default: `residue`
  - Example: `-mode0 whole -mode1 residue`

  Defines how the selected regions (`-sel0`, `-sel1`) are partitioned for distance calculations.

- **-pbc <true or false>**
  - Default: `false`
  - Example: `-pbc true`

  Specifies whether periodic boundary conditions should be applied during distance calculations.

- **-dt <time interval>**
  - Default: `1.0`
  - Example: `-dt 0.1`

  Sets the time interval (first column of the output) for the time series data.

- **-distance_type <standard or minimum or intra>**†
  - Default: `standard`
  - Example: `-distance_type standard`

  Specifies the type of distance to compute:
  - **standard**: Standard distance between molecules.
  - **minimum**: Minimum atomic distance between regions.
  - **intra**: Intra-molecular distances.

  The `intra` option is useful when evaluating distances between atoms within the same molecule. For instance, in a system with 100 identical polymers (resname `PLM`) having terminal atoms named `CP1` and `CP2`, computing the end-to-end distance within each polymer requires care. Using `-distance_type standard` with `-sel0 name CP1` and `-sel1 name CP2` would also include inter-polymer distances. In contrast, `-distance_type intra` restricts calculations to within individual molecules.

- **-mindist_type1 <site or com>**†
  - Default: `site`
  - Example: `-mindist_type0 com -mindist_type1 site`

Specifies the candidate sites for minimum distance calculations:

- **site**: All atoms in the region are considered.
- **com**: Only the center of mass is considered.

When both regions use `com`, the result is equivalent to standard center-of-mass distance.

3.4.2 Output File Format

- `*.dis` file

  This file contains distances (or minimum distances) between the selected regions. Suppose `-mode0` and `-mode1` divide selections into n_1 and n_2 molecules, respectively. Then $n_1 \times n_2$ distances will be recorded per frame. The format is as follows:

  - Column 1: Time
  - Column 2: Distance between molecule pair (1, 1)
  - Column 3: Distance between pair (1, 2)
    …
  - Column n_2: Distance between pair (1, n_2)
  - Column $n_2 + 1$: Distance between pair (2, 1)
  - Column $n_2 + 2$: Distance between pair (2, 2)
    …

  All distances are reported in Å.

3.4.3 Example of Analysis

3.4.3.1 Standard Atom-to-Region Distance

The following example computes distances between a protein (`resid 1 to 192`) and 10 ligand molecules (`resname LIG`):

```
#!/bin/bash

anatra distance                                  \
  -stype          parm7                          \
  -sfile          complex.prmtop                 \
  -tintype        dcd                            \
  -tin            INP.dcd                        \
  -fhead          out                            \
  -pbc            true                           \
```

(continues on next page)

(continued from previous page)

```
-distance_type  standard                    \
-dt             0.1                         \
-sel0           resid 1 to 192              \
-sel1           resname LIG                 \
-mode0          whole                       \
-mode1          residue
```

3.5 Center of Mass Analysis and Mean Square Displacement (center_of_mass, com)

The program `center_of_mass` (`com`) computes the temporal variation of molecular centers of mass and the mean square displacement (MSD). To support two-dimensional diffusion (lateral diffusion) within lipid membranes, it also allows for MSD calculations restricted to the xy-plane.

3.5.1 Theoretical Background

3.5.1.1 Mean Square Displacement

Here, we introduce the definition of the mean square displacement (MSD) $\chi(t)$ and discuss its significance based on the diffusion equation.

Let $\mathbf{r}(t)$ denote the position of a molecule at time t. The MSD is defined as follows:

$$\chi\left(t\right) = \left\langle \left|\mathbf{r}\left(t\right) - \mathbf{r}\left(0\right)\right|^2 \right\rangle . \tag{3.1}$$

That is, $\chi(t)$ represents the time-averaged squared displacement of a molecule after a time interval t. To understand how MSD is described by the diffusion equation, let us consider a probability density $\rho(\mathbf{r}, t)$ representing the location of a molecule, and a diffusion coefficient D. The diffusion equation is written as

$$\frac{\partial}{\partial t}\rho\left(\mathbf{r},t\right) = D\nabla^2\rho\left(\mathbf{r},t\right). \tag{3.2}$$

Assuming that the molecule was initially located at $\mathbf{0}$ at time $t = 0$, the initial condition is given by

$$\rho\left(\mathbf{r}, t=0\right) = \delta\left(\mathbf{r}\right) \tag{3.3}$$

Solving the diffusion equation under this condition yields

$$\rho\left(\mathbf{r},t\right) = \frac{1}{\left(4\pi Dt\right)^{3/2}} \exp\left[-\frac{\left|\mathbf{r}\right|^2}{4Dt}\right]. \tag{3.4}$$

Using this solution, the MSD can be expressed as

$$\chi\left(t\right) = \int d\mathbf{r}\ \left|\mathbf{r}\right|^2 \rho\left(\mathbf{r},t\right) \tag{3.5}$$

$$= 6Dt, \tag{3.6}$$

indicating that MSD increases linearly with time, with a slope of $6D$. Because the diffusion equation is valid only in the long-time limit, the diffusion coefficient should be defined as

$$D = \frac{1}{6} \lim_{t \to \infty} \frac{\left\langle \left| \mathbf{r}(t) - \mathbf{r}(0) \right|^2 \right\rangle}{t}, \tag{3.7}$$

This expression is known as Einstein′s relation. It indicates that the diffusion coefficient can be computed from the slope of the MSD in the long-time regime.

3.5.2 List of Options

Option	Description	Option	Description
`-stype`	Common	`-sfile`	Common
`-tintype`	Common	`-tin`	Common
`-flist_traj`	Common	`-fhead`	Common
`-sel0`	Common	`-mode`	Common
`-outmsd`	Mode specific	`-unwarp`	Mode specific
`-msddim`	Mode specific	`-dt`	Mode specific
`-t_sparse`	Mode specific	`-t_range`	Mode specific
`-prep_only`	Common		

- **-outmsd <true or false>**
  - Default: `false`
  - Example: `-outmsd true`
  - Note: If set to `true`, both `-msddim` and `-msdrange` must also be specified.

  Specifies whether or not to compute the MSD.

- **-unwrap <true or false>**
  - Default: `false`
  - Example: `-unwrap false`

  Specifies whether to unwrap the trajectory. Since wrapped trajectories cannot yield accurate MSD values, it is necessary to set this option to `true`. Note that trajectories segmented by periodic boundary conditions (PBC) — as in GROMACS — are not supported by this option. In such cases, preprocessing with GROMACS `trjconv` is required.

- **-msddim <2 or 3>**
  - Default: `3`

  - Example: `-msddim 3`

  Specifies the dimensionality of the MSD calculation. Use `3` for general diffusion analysis and `2` for lateral diffusion within lipid bilayers.

- **-dt**
  - Default: `0.1`
  - Example: `-dt 0.1`

  Specifies the time interval between snapshots.

- **-t_range**
  - Default: `10`
  - Example: `-t_range 10`

  Specifies the time scale over which MSD is computed.

- **-t_sparse**

  Specifies the time interval for MSD output. For example, with `-dt 0.1` ($= \delta t$) and `-t_sparse 1` ($= \Delta t$), the MSD (in Å^2) is computed at time intervals:

$$\Delta T = \text{nint}\left(\frac{\Delta t}{\delta t}\right)\delta t \tag{3.8}$$

3.5.3 Output File Formats

- `*.com`

  This file contains the center-of-mass coordinates of each molecule within the selected region, in `xyz` format.

- `*.msd`

  This file stores the MSD values for individual molecules. Output is generated only when `-outmsd true` is specified.
  - Column 1: Time
  - Column 2: MSD of molecule 1
  - Column 3: MSD of molecule 2

  …

- `*.msdave`

  This file contains the averaged MSD across all molecules. Output is generated only when `-outmsd true` is specified.
  - Column 1: Time

- Column 2: Mean MSD
- Column 3: Standard deviation of MSD

3.5.4 Examples of Analysis

3.5.4.1 MSD Calculation for a Protein

The following is an example of calculating the MSD of a single protein (`resid 1 to 192`) dissolved in aqueous solution. Since the protein consists of multiple amino acid residues, it is necessary to specify the mode as "`-mode whole`".

```
#!/bin/bash

anatra center_of_mass        \
  -stype     parm7           \
  -sfile     complex.prmtop  \
  -tintype   dcd             \
  -tin       INP.dcd         \
  -fhead     out             \
  -outmsd    true            \
  -sel0      resid 1 to 192  \
  -mode      whole           \
  -msddim    3               \
  -dt        0.01            \
  -t_range   100             \
  -t_sparse  0.1
```

3.5.4.2 MSD Calculation for Multiple Small Molecules

Consider a system containing 100 ligand molecules (`resname LIG`) dissolved in aqueous solution. The following example computes both the MSD of each molecule and their average. In this case, molecular boundaries can be determined based on residue numbers, so the mode is set to "`-mode residue`".

```
#!/bin/bash

anatra center_of_mass        \
  -stype     parm7           \
  -sfile     complex.prmtop  \
  -tintype   dcd             \
  -tin       INP.dcd         \
  -fhead     out             \
  -outmsd    true            \
```

(continues on next page)

(continued from previous page)

```
-sel0      resname LIG     \
-mode      residue         \
-msddim    3               \
-dt        0.01            \
-t_range   100             \
-t_sparse  0.01
```

3.6 Radial Distribution Function (radial_distr, rdf)

3.6.1 Theoretical Background

This section explains the definition of the radial distribution function (RDF).

3.6.1.1 RDF for Identical Particles

Consider a system containing N_a particles of type a, with a total volume V and a bulk number density $\rho_a = N_a/V$. The local density field $\hat{\rho}_a(\mathbf{r})$ is defined as

$$\hat{\rho}_a(\mathbf{r}) = \sum_{i=1}^{N_a} \delta(\mathbf{r} - \mathbf{r}_i). \tag{3.9}$$

Here, $\mathbf{r}_i$ is the position of the i-th particle of type a. If no external field is present, the ensemble average of the local density field is reduced to the bulk density.

$$\langle \hat{\rho}_a(\mathbf{r}) \rangle = \rho_a. \tag{3.10}$$

The, let us define the two-body density correlation function as

$$\chi(\mathbf{r}, \mathbf{r}') = \langle \hat{\rho}_a(\mathbf{r}) \hat{\rho}_a(\mathbf{r}') \rangle = \left\langle \sum_{i=1}^{N_a} \sum_{j=1}^{N_a} \delta(\mathbf{r} - \mathbf{r}_i) \delta(\mathbf{r}' - \mathbf{r}_j) \right\rangle. \tag{3.11}$$

Separating the terms for $i = j$ and $i \neq j$ gives

$$\chi(\mathbf{r}, \mathbf{r}') = \sum_{i=1}^{N_a} \langle \delta(\mathbf{r} - \mathbf{r}_i) \delta(\mathbf{r}' - \mathbf{r}_i) \rangle + \sum_{i=1}^{N_a} \sum_{j \neq i}^{N_a} \langle \delta(\mathbf{r} - \mathbf{r}_i) \delta(\mathbf{r}' - \mathbf{r}_j) \rangle. \tag{3.12}$$

The first term represents the probability that the same particle exists at both $\mathbf{r}$ and $\mathbf{r}'$, which is clearly zero unless $\mathbf{r} = \mathbf{r}'$, i.e.,

$$\sum_{i=1}^{N_a} \langle \delta(\mathbf{r} - \mathbf{r}_i) \delta(\mathbf{r}' - \mathbf{r}_i) \rangle = \rho_a \delta(\mathbf{r} - \mathbf{r}'). \tag{3.13}$$

The second term, called the two-body density, represents the probability that one particle is at $\mathbf{r}$ and another is at $\mathbf{r}'$:

$$\rho_{aa}^{(2)}(\mathbf{r}, \mathbf{r}') = \sum_{i=1}^{N_a} \sum_{j \neq i}^{N_a} \langle \delta(\mathbf{r} - \mathbf{r}_i) \delta(\mathbf{r}' - \mathbf{r}_j) \rangle. \tag{3.14}$$

If $\mathbf{r}$ and $\mathbf{r}'$ are far apart, their correlation is negligible. Thus

$$\rho_{aa}^{(2)}(\mathbf{r},\mathbf{r}') \approx \rho_a^2\left(1-\frac{1}{N_a}\right). \qquad (|\mathbf{r}-\mathbf{r}'|\to\infty) \tag{3.15}$$

The pair distribution function is defined as

$$g_{aa}^{(2)}(\mathbf{r},\mathbf{r}') = \frac{1}{\rho_a^2(1-1/N_a)}\rho_{aa}^{(2)}(\mathbf{r},\mathbf{r}'). \tag{3.16}$$

For large systems, $1/N_a \approx 0$. Therefore,

$$g_{aa}^{(2)}(\mathbf{r},\mathbf{r}') = \frac{1}{\rho_a^2}\rho_{aa}^{(2)}(\mathbf{r},\mathbf{r}'). \tag{3.17}$$

In uniform systems, the function depends only on the relative position, indicating

$$g_{aa}^{(2)}(\mathbf{r},\mathbf{r}') = g_{aa}^{(2)}(\mathbf{0},\mathbf{r}'-\mathbf{r}). \tag{3.18}$$

Let $\mathbf{r}$ be $\mathbf{r}'-\mathbf{r}$ and $g_{aa}(\mathbf{r})$ be the spatial distribution function (SDF) defined as

$$g_{aa}(\mathbf{r}) = g_{aa}^{(2)}(\mathbf{0},\mathbf{r}). \tag{3.19}$$

Averaging over orientation leads to the radial distribution function (RDF) defined as

$$g_{aa}(r) = \frac{1}{4\pi r^2}\int d\mathbf{r}\,\delta(|\mathbf{r}|-r)\,g_{aa}(\mathbf{r}). \tag{3.20}$$

For isotropic systems, the following relationship holds.

$$g_{aa}(|\mathbf{r}|) = g_{aa}(\mathbf{r}). \tag{3.21}$$

RDF gives the relative probability of finding a particle at distance r compared to bulk. $g_{aa}(r) > 1$ indicates higher probability, $g_{aa}(r) < 1$ indicates lower probability than bulk.

3.6.1.2 RDF for Heterogeneous Particles

Consider a system with N_a particles of type a and N_b of type b, with respective bulk densities $\rho_a = N_a/V$ and $\rho_b = N_b/V$. The two-body density is

$$\rho_{ab}^{(2)}(\mathbf{r},\mathbf{r}') = \sum_{i=1}^{N_a}\sum_{j=1}^{N_b}\left\langle\delta(\mathbf{r}-\mathbf{r}_i^a)\delta(\mathbf{r}'-\mathbf{r}_j^b)\right\rangle. \tag{3.22}$$

For uncorrelated particles,

$$\rho_{ab}^{(2)}(\mathbf{r},\mathbf{r}') \approx \rho_a\rho_b. \tag{3.23}$$

The spatial distribution function is

$$g_{ab}(\mathbf{r}) = \frac{1}{\rho_a\rho_b}\rho_{ab}^{(2)}(\mathbf{0},\mathbf{r}), \tag{3.24}$$

and the RDF is

$$g_{ab}(r) = \frac{1}{4\pi r^2}\int d\mathbf{r}\,\delta(|\mathbf{r}|-r)g_{ab}(\mathbf{r}). \tag{3.25}$$

3.6.2 Option List

Option	Description	Option	Description
`-stype`	Common	`-sfile`	Common
`-tintype`	Common	`-tin`	Common
`-flist_traj`	Common	`-fhead`	Common
`-sel0`	Common	`-sel1`	Common
`-mode0`	Common	`-mode1`	Common
`-dr`	Mode specific	`-identical`	Mode specific
`-normalize`	Mode specific	`-separate_self`	Mode specific

- **-dr**
  - Default: `0.1`

  Specifies the bin width for RDF calculation.

- **-identical**
  - Default: `false`

  Set to `true` when `-sel0` and `-sel1` are the same. As discussed in the theory section, RDF calculations differ depending on whether the particles are identical or not, so this must be specified by the user.

- **-normalize**
  - Default: `true`

  Specifies whether to output the RDF $g_{12}(r)$ directly (`true`), or multiply by the average number density ρ_2 of `sel1` to output $\rho_{12}(r) = \rho_2 g_{12}(r)$ (`false`).

- **-separate_self**
  - Default: `false`

  Determines whether to separate intra- and intermolecular contributions in RDF. This is useful, for example, when calculating RDF between different atoms in the same type of molecule (e.g., OW-HW1 in water). Setting `true` separates contributions from within the same molecule.

3.6.3 Output File Format

- `*.rdf` file

  This file contains the RDF output. The content varies depending on the `-separate_self` option:

  - If `-separate_self false`:
    - Column 1: Distance
    - Column 2: RDF
  - If `-separate_self true`:
    - Column 1: Distance
    - Column 2: Intermolecular RDF
    - Column 3: Intramolecular RDF (self-correlation function)

3.6.4 Examples of Analysis

3.6.4.1 RDF between Oxygen Atoms of Water Molecules

```
#!/bin/bash

anatra rdf                                      \
  -stype         parm7                          \
  -sfile         complex.prmtop                 \
  -tintype       dcd                            \
  -tin           INP.dcd                        \
  -fhead         out                            \
  -sel0          resname WAT and name O         \
  -sel1          resname WAT and name O         \
  -mode0         residue                        \
  -mode1         residue                        \
  -dr            0.1                            \
  -identical     true                           \
  -normalize     true                           \
  -separate_self false
```

3.6.4.2 RDF between Oxygen and Hydrogen Atoms in Water Molecules

```
#!/bin/bash

anatra rdf                                      \
  -stype         parm7                          \
  -sfile         complex.prmtop                 \
  -tintype       dcd                            \
  -tin           INP.dcd                        \
  -fhead         out                            \
  -sel0          resname WAT and name O         \
  -sel1          resname WAT and name H1        \
  -mode0         residue                        \
  -mode1         residue                        \
  -dr            0.1                            \
  -identical     false                          \
  -normalize     true                           \
  -separate_self true
```

3.6.4.3 RDF between Protein Cα Atoms and Water Oxygen Atoms

This example calculates the RDF between the Cα atoms (`resid 1 to 192`) of a protein and oxygen atoms of water molecules, and outputs the average of the individual RDFs.

```
anatra rdf                                      \
  -stype         parm7                          \
  -sfile         complex.prmtop                 \
  -tintype       dcd                            \
  -tin           INP.dcd                        \
  -fhead         out                            \
  -sel0          resid 1 to 192 and name CA     \
  -sel1          resname WAT and name O         \
  -mode0         atom                           \
  -mode1         residue                        \
  -dr            0.1                            \
  -identical     false                          \
  -normalize     true                           \
  -separate_self false
```

3.7 Spatial Distribution Function (`spatial_distr`, `sdf`)

3.7.1 Theoretical Background

Consider a system with N_a particles of type a, volume V, and number density $\rho_a = N_a/V$. Assume that one molecule is fixed at the center of the system, including its orientation, acting effectively as an external field. Although such a situation does not naturally occur in standard solution-phase MD simulations, a virtually equivalent condition can be constructed by aligning all trajectories to a reference structure (fitting), as described at the end of this section.

Similar to the radial distribution function, we define the local number density of particle a as

$$\hat{\rho}_a(\mathbf{r}) = \sum_{i=1}^{N_a} \delta(\mathbf{r} - \mathbf{r}_i). \tag{3.26}$$

Its ensemble average under the external field gives the one-body density

$$\rho_a(\mathbf{r}) = \langle \hat{\rho}_a(\mathbf{r}) \rangle_{\text{ext}} = \left\langle \sum_{i=1}^{N_a} \delta(\mathbf{r} - \mathbf{r}_i) \right\rangle_{\text{ext}}. \tag{3.27}$$

Here, $\langle \cdots \rangle_{\text{ext}}$ denotes the ensemble average under the external field. In the absence of an external field or far from the fixed molecule, $\rho_a(\mathbf{r}) = \rho_a$. THen, we define the spatial distribution function (SDF) as:

$$g_a(\mathbf{r}) = \frac{\rho_a(\mathbf{r})}{\rho_a}. \tag{3.28}$$

If $g_a(\mathbf{r}) > 1$, the local probability of finding particle a is larger than the bulk value; if $g_a(\mathbf{r}) < 1$, it is lower than in bulk.

To compute the SDF, the system is first fitted such that the molecule treated as the external field overlaps with a reference structure. For unwrapped trajectories, PBC wrapping must be performed in advance (Fig. 3.2). The `spatial_distr` mode handles all these steps internally. Once transformed, the reference molecule is fixed in position and orientation, making it effectively act as a static field, and the surrounding molecular distribution can be analyzed as an SDF. If the reference molecule lacks a stable structure, fitting becomes unreliable and SDF computation is infeasible. However, for proteins with stable native structures and minimal conformational changes, SDF analysis is applicable.

3.7.2 List of Options

Option	Remarks	Option	Remarks
`-stype`	Common	`-sfile`	Common
`-tintype`	Common	`-tin`	Common
`-flist_traj`	Common	`-fhead`	Common
`-selX`	Common	`-mode`	Common
`-ng3`	Mode specific	`-del`	Mode specific
`-origin`	Mode specific	`-fit`	Mode specific
`-refpdb`	Mode specific	`-refselid`	Mode specific
`-fitselid`	Mode specific	`-use_spline`	Mode specific
`-spline_resolution`	Mode specific	`-prep_only`	Common

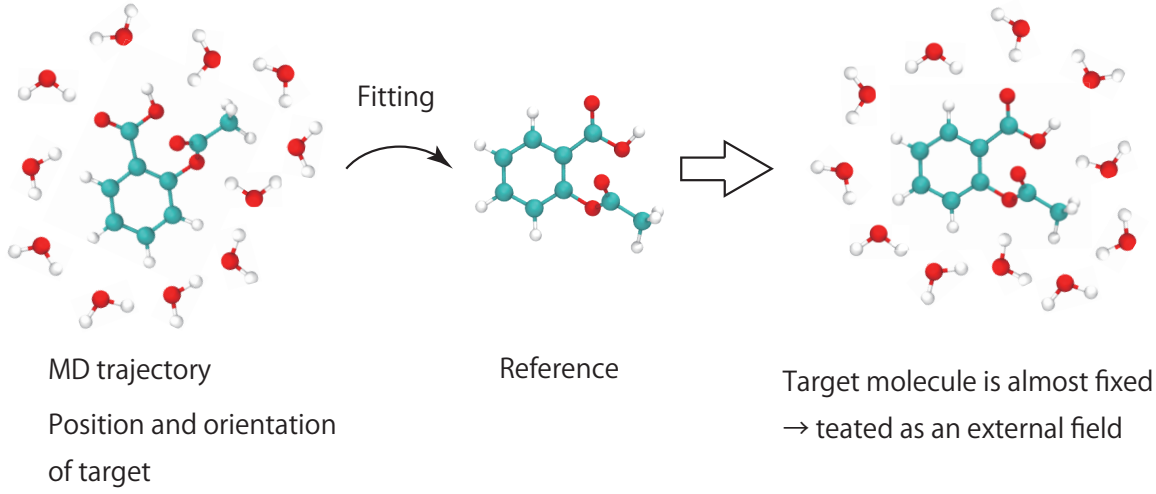

Fig. 3.2: By aligning the system such that the molecule of interest overlaps with the reference structure (fitting), it becomes possible to examine the three-dimensional distribution of surrounding molecules relative to a fixed molecular orientation. The `spatial_distr` mode automates this entire procedure.

Advanced options for conditional SDF analysis exist but will be officially supported in a future update.

- **-ng3 <number of grid points along x, y, z>**
  - Default: `50 50 50`
  - Example: `-ng3 50 50 50`

  Specifies the number of grid points along each spatial axis, n_x, n_y, n_z. The total number of grid points is $n_x n_y n_z$.

- **-del <grid spacing along x, y, z axes (Å)>**
  - Default: `0.5 0.5 0.5`
  - Example: `-del 0.5 0.5 0.5`

  Specifies the grid spacing $\Delta x, \Delta y, \Delta z$ in Å. The volume element per grid point is $\Delta V = \Delta x \Delta y \Delta z$.

- **-origin <coordinates of grid origin (x, y, z) in Å>**
  - Default: `0.0 0.0 0.0`
  - Example: `-origin 0.0 0.0 0.0`

Sets the origin of the grid. For origin (x_0, y_0, z_0), the coordinates of grid point (i_x, i_y, i_z) are:

$$x_i = x_0 + \Delta x\,(i_x - 1) \tag{3.29}$$

$$y_i = y_0 + \Delta y\,(i_y - 1) \tag{3.30}$$

$$z_i = z_0 + \Delta z\,(i_z - 1) \tag{3.31}$$

- **-fit <true or false>**
  - Default: `false`
  - Example: `-fit true`

  If set to `true`, the system is PBC-wrapped and then fitted so that the solute molecule (treated as the external field) aligns with the reference structure. The following must be specified:
  - `-refpdb`
  - `-refselid`
  - `-fitselid`
  - `-sel0` (target for SDF calculation)
  - `-sel1` (reference fitting region)
  - `-sel2` (trajectory fitting region)

  Important: If the input trajectory has already been fitted, you must explicitly set `-fit false`. Applying PBC wrapping after fitting leads to invalid configurations.

- **-refselid <selid>**
  - Default: N/A
  - Example: `-refselid 1`

  Specifies the selid of the reference region used for fitting in the reference structure. For example, if using `-sel1` as the reference selection, set `-refselid 1`.

- **-fitselid <selid for fitting region in trajectory>**
  - Default: N/A
  - Example: `-fitselid 2`

  Specifies the selid for the region used for fitting within the trajectory. For example, if using `-sel2` for trajectory fitting, set `-fitselid 2`.

- **-use_spline <true or false>**

  - Default: `false`
  - Example: `-use_spline true`
  - Note: The resolution is controlled by `-spline_resolution`.

  Applies spline interpolation (Akima spline) to the SDF grid.

- **-spline_resolution <integer resolution factor>**
  - Default: `4`
  - Example: `-spline_resolution 4`
  - Note: Active only if `-use_spline true` is specified.

  Specifies the subdivision factor n_r for each grid axis. Given the original grid spacing $\Delta\alpha$ ($\alpha = x, y, z$), the refined grid spacing becomes:

$$\Delta\alpha' = \frac{\Delta\alpha}{n_r} \tag{3.32}$$

  The total number of grid points changes from N to:

$$N' = n_r^3 \cdot N \tag{3.33}$$

3.7.3 Output File Format

- `*_out_g_original.dx`
  SDF output in DX format. No spline interpolation applied.
- `*_out_g_spline.dx`
  SDF output in DX format with spline interpolation. Generated only if `-use_spline true` is specified.

3.7.4 Examples of Analysis

3.7.4.1 SDF of Water Molecules Around a Protein

The following script computes the SDF of water molecule centers of mass around a protein (`resid 1 to 192`):

```
#!/bin/bash

anatra spatial_distr                        \
  -stype             parm7                  \
  -sfile             complex.prmtop         \
```

(continues on next page)

(continued from previous page)

```
-tintype           dcd                    \
-tin               INP.dcd                \
-fhead             out                    \
-sel0              water                  \
-sel1              resid 1 to 192 and noh  \
-sel2              resid 1 to 192 and noh  \
-mode              residue                \
-ng3               50 50 50               \
-del               0.4 0.4 0.4            \
-origin            -10.0 -10.0 -10.0      \
-fit               true                   \
-refpdb            ref.pdb                \
-refselid          1                      \
-fitselid          2                      \
-use_spline        true                   \
-spline_resolution 4
```

3.8 Root-Mean-Square Deviation (`rmsd`)

3.8.1 Theoretical Background

This section explains the definition of the root-mean-square deviation (RMSD). RMSD quantitatively evaluates the structural deviation of a molecule or a specific region of interest from a reference structure at a given time during an MD simulation. It is commonly used to assess how much a structure deviates from, for example, an experimental crystal structure during the equilibration phase, and whether or not it relaxes over time.

Consider a molecule composed of N_{sel} atoms. Let $\mathbf{r}_i$ denote the position of the i-th atom, $\mathbf{r}_i^{\mathrm{ref}}$ the corresponding coordinate in the reference structure, and $\mathbf{r}_i(t)$ the position at time t. Then, the RMSD $\chi(t)$ is defined as:

$$\chi(t) = \sqrt{\frac{1}{N_{\mathrm{sel}}} \sum_{i=1}^{N_{\mathrm{sel}}} \left|\mathbf{r}_i(t) - \mathbf{r}_i^{\mathrm{ref}}\right|^2}. \tag{3.34}$$

3.8.2 List of Options

Option	Remarks	Option	Remarks
`-stype`	Common	`-sfile`	Common
`-tintype`	Common	`-tin`	Common
`-fout`	Common	`-sel0`	Common
`-sel1`	Common	`-sel2`	Common
`-sel3`	Common	`-fit`	Mode specific
`-refpdb`	Common	`-fitselid`	Mode specific
`-refselid`	Mode specific	`-rmsdselid`	Mode specific
`-rmsdrefselid`	Mode specific		

- **-fout <output file name>**
  - Default: `out.rmsd`
  - Example: `-fout out.rmsd`

  Specifies the filename for the output of RMSD results.

- **-refpdb <reference structure in PDB format>**
  - Default: N/A
  - Example: `-refpdb ref.pdb`

  Specifies the reference structure used for RMSD calculation.

- **-fit <true or false>**
  - Default: `false`
  - Example: `-fit true`
  - Note: If set to `true`, `-refselid` and `-fitselid` must be specified.

  Indicates whether to perform structural fitting prior to RMSD calculation.

- **-fitselid <selid>**
  - Default: N/A
  - Example: `-fitselid 1`
- **-refselid <selid>**

  - Default: N/A
  - Example: `-refselid 2`

  Specify the selection IDs of the fitting regions in the trajectory and the reference structure, respectively. These options are required when `-fit true` is set.

- **-rmsdselid <selid>**
  - Default: N/A
  - Example: `-rmsdselid 2`

  Specifies the selid corresponding to the region used in the trajectory for RMSD calculation.

- **-rmsdrefselid <selid>**
  - Default: N/A
  - Example: `-rmsdrefselid 3`

  Specifies the selid corresponding to the region in the reference structure for RMSD calculation.

3.8.3 Output File Format

- `*.rmsd` file

  This file contains the time evolution of RMSD values.
  - Column 1: Time
  - Column 2: RMSD

3.8.4 Examples of Analysis

3.8.4.1 RMSD from the Crystal Structure of a Protein in Solution

This example calculates the RMSD of the Cα atoms of a protein (`resid 1 to 161`) with respect to a crystal structure.

```
anatra rmsd                                  \
  -stype        parm7                        \
  -sfile        complex.prmtop               \
  -tintype      dcd                          \
  -tin          INP.dcd                      \
  -fout         out.rmsd                     \
  -sel0         resid 1 to 161 and name CA   \
  -sel1         resid 1 to 161 and name CA   \
```

(continues on next page)

(continued from previous page)

```
-sel2         resid 1 to 161 and name CA    \
-sel3         resid 1 to 161 and name CA    \
-fit          true                          \
-refpdb       ref.pdb                       \
-fitselid     0                             \
-refselid     1                             \
-rmsdselid    2                             \
-rmsdrefselid 3
```

3.9 Interaction Energy (`interaction_energy`, `ene`)

The `interaction_energy` (`ene`) mode allows the calculation of interaction energies between selected molecules, groups of molecules, or specific regions. Currently, **ANATRA** supports the following three types of interactions:

- **Electrostatic Interaction**
  Electrostatic interaction energy U_{elec} between molecules A and B can be computed using two schemes. The first is the bare interaction,

$$U_{\mathrm{elec}} = \sum_{i \in \mathrm{A}} \sum_{j \in \mathrm{B}} \frac{q_i q_j}{r_{ij}}, \tag{3.35}$$

  where i and j denote atomic indices in molecules A and B, respectively. The second scheme uses the Smooth Particle Mesh Ewald (SPME) method, which is the standard approach for accounting for long-range electrostatic interactions in MD simulations. If consistency with MD simulations is required, the SPME method should be employed.

- **Lennard-Jones (LJ) Interaction**
  The LJ interaction, which empirically describes van der Waals interactions, is given by

$$U_{\mathrm{LJ}} = \sum_{i \in \mathrm{A}} \sum_{j \in \mathrm{B}} 4\epsilon_{ij} \left[\left(\frac{\sigma_{ij}}{r_{ij}} \right)^{12} - \left(\frac{\sigma_{ij}}{r_{ij}} \right)^{6} \right]. \tag{3.36}$$

  Here, ϵ_{ij} and σ_{ij} are the LJ parameters between atom i in molecule A and atom j in molecule B.

- **Attractive LJ Interaction**
  The attractive component of the LJ interaction is defined as:

$$U_{\mathrm{attr}} = \sum_{i \in \mathrm{A}} \sum_{j \in \mathrm{B}} u_{\mathrm{attr},ij}, \tag{3.37}$$

$$u_{\mathrm{attr},ij} = \begin{cases} 4\epsilon_{ij} \left[\left(\frac{\sigma_{ij}}{r_{ij}} \right)^{12} - \left(\frac{\sigma_{ij}}{r_{ij}} \right)^{6} \right] & r_{ij} \geq 2^{1/6}\sigma, \\ -\epsilon_{ij} & r_{ij} < 2^{1/6}\sigma. \end{cases} \tag{3.38}$$

  This formulation removes the repulsive part of the LJ potential, eliminating the multivalued nature of energy with respect to distance (see Fig. 3.3). When constructing a PMF for molecular binding, using the attractive LJ interaction as a reaction coordinate can more clearly distinguish between bound and unbound states than the standard LJ potential.

In addition to the individual components above, the sum of electrostatic and LJ can also be calculated.

3.9.1 List of Options

Option	Remarks	Option	Remarks
-stype	Common	-sfile	Common
-tintype	Common	-tin	Common
-flist_traj	Common	-fhead	Common
-sel0	Common	-sel1	Common
-mode0	Common	-mode1	Common
-parmformat	Mode specific	-fanaparm	Mode specific
-pbc	Mode specific	-calc_vdw	Mode specific
-calc_elec	Mode specific	-vdw_type	Mode specific
-elec_type	Mode specific	-dt	Mode specific
-rvdwcut	Mode specific	-relcut	Mode specific
-pme_alpha	Mode specific	-pme_grids	Mode specific
-pme_rigid	Mode specific	-pme_dual	Mode specific
-pme_fprm	Mode specific	-prep_only	Common

- **-parmformat <prmtop or anaparm>**
  - Default: `prmtop`
  - Example: `-parmformat anaparm`
  - Note: When set to `anaparm`, `-fanaparm` must also be specified.

  Specifies the parameter file for atomic charges. When using `prmtop`, the file given by `-sfile` will be automatically used.

- **-fanaparm <filename>**
  - Default: N/A
  - Example: `-fanaparm complex.anaparm`

  Specifies the `anaparm` file containing LJ parameters and atomic charges. Required when `-parmformat anaparm` is used.

- **-pbc <true or false>**
  - Default: `false`
  - Example: `-pbc true`

  Enables periodic boundary condition treatment, including image interactions from adjacent periodic boxes.

- **-calc_vdw <true or false>**
  - Default: `true`
  - Example: `-calc_vdw false`

  Specifies whether to compute van der Waals (LJ) interactions.

- **-calc_elec <true or false>**
  - Default: `false`
  - Example: `-calc_elec true`

  Specifies whether to compute electrostatic interactions.

- **-vdw_type <standard or attractive>**
  - Default: `standard`
  - Example: `-vdw_type attractive`

  Specifies which LJ interaction function to use.
  `standard` computes the standard LJ (Eq. \eqref{LJ}), and `attractive` computes the attractive LJ (Eq. \eqref{Uattr}).

- **-elec_type <bare or pme>**
  - Default: `bare`
  - Example: `-elec_type bare`

  Specifies the method for electrostatic interaction calculation:
  `bare` for Eq. \eqref{elec}, `pme` for the PME method.

- **-dt <time interval>**
  - Default: `0.1`
  - Example: `-dt 1.0`

  Sets the time resolution (first column) of the time-series output.

- **-rvdwcut <cutoff distance (Å)>**
  - Default: `12.0`
  - Example: `-rvdwcut 20.0`

  Cutoff distance for van der Waals interactions.

- **-relcut <cutoff distance (Å)>**
  - Default: `1.0e10`
  - Example: `-relcut 1.0e10`

  Cutoff distance for electrostatic interactions.

- **-pme_alpha <PME screening parameter ($Å^{-1}$)>**
  - Default: `0.35`
  - Example: `-pme_alpha 0.35`

  Specifies the screening parameter used in PME. Only used when `-elec_type pme`.

- **-pme_grids <grid size in x, y, z>**
  - Default: `64 64 64`
  - Example: `-pme_grids 64 64 64`

  Grid size n_x, n_y, n_z used in PME. Total number of grid points is $n_x n_y n_z$.

- **-pme_rigid <true or false>**
  - Default: `false`
  - Example: `-pme_rigid false`

  If the region specified by `-sel0` is spatially fixed, set to `true` to reduce PME computational cost.

3.9.2 Output File Format

- `*.uij` file

  This file contains pairwise interaction energies between selected regions. If selections 0 and 1 are divided into n_1 and n_2 molecules, respectively (via `-mode0` and `-mode1`), the output contains $n_1 n_2$ interaction terms:

  - Column 1: Time
  - Column 2: Interaction between pair (1, 1)
  - Column 3: Interaction between pair (1, 2)
    …
  - Column n_2: Interaction between pair (1, n_2)
  - Column $n_2 + 1$: Interaction between pair (2, 1)
    …

3.9.3 Examples of Analysis

The following are minimal examples for common types of interaction energy analysis.

3.9.3.1 Electrostatic + LJ Interaction Between Solute Molecules

This example computes the total of electrostatic and LJ interactions between a protein (`resid 1 to 161`) and ligand molecules (`resname LIG`), using the SPME method for electrostatics:

```
#!/bin/bash

anatra energy                                \
  -stype       parm7                         \
  -sfile       complex.prmtop                \
  -tintype     dcd                           \
  -tin         INP.dcd                       \
  -fhead       out                           \
  -parmformat  prmtop                        \
  -pbc         true                          \
  -calc_vdw    true                          \
  -calc_elec   true                          \
  -vdw_type    standard                      \
  -elec_type   pme                           \
  -mode0       whole                         \
  -mode1       residue                       \
  -dt          0.1                           \
  -rvdwcut     12.0                          \
  -relcut      12.0                          \
  -pme_alpha   0.35                          \
  -pme_grids   64 64 64                      \
  -pme_rigid   false                         \
  -sel0        resid 1 to 161                \
  -sel1        resname LIG                   \
  -prep_only   false
```

3.9.3.2 LJ Interaction Between Solute Molecules

This example computes only the LJ interactions between a protein and ligand molecules:

```
#!/bin/bash

anatra energy                                \
  -stype       parm7                         \
  -sfile       complex.prmtop                \
  -tintype     dcd                           \
  -tin         INP.dcd                       \
  -fhead       out                           \
  -parmformat  prmtop                        \
  -pbc         true                          \
  -calc_vdw    true                          \
  -calc_elec   false                         \
  -vdw_type    standard                      \
  -mode0       whole                         \
  -mode1       residue                       \
  -dt          0.1                           \
  -rvdwcut     12.0                          \
  -relcut      1.0e10                        \
  -sel0        resid 1 to 161                \
  -sel1        resname LIG                   \
  -prep_only   false
```

3.10 Lipid Order Parameter (`lipid_order`, `scd`)

The `lipid_order` (`scd`) mode allows for the calculation of the orientational order parameter S_{CD} associated with acyl chains of phospholipids. Using the membrane normal direction as the z-axis, the angle θ is defined between this axis and the vector connecting a specific carbon atom and its bonded hydrogen. The S_{CD} parameter is then defined as:

$$S_{\mathrm{CD}} = \frac{1}{2} \left\langle 3\cos^2\theta - 1 \right\rangle . \tag{3.39}$$

The range of S_{CD} is $0 \leq S_{\mathrm{CD}} \leq 1$, where higher values indicate greater order, and a value of 0 corresponds to complete disorder. Since S_{CD} is defined for each carbon atom, this analysis provides position-resolved information on lipid order along the acyl chains.

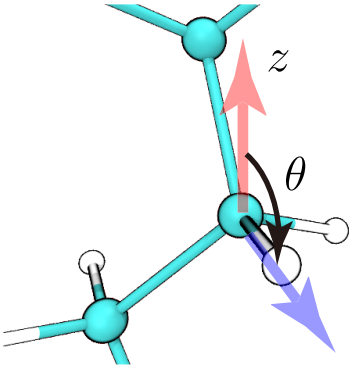

Fig. 3.3: The angle θ is defined between the membrane normal (the z-axis) and the vector connecting a specific carbon and its bonded hydrogen atom. This angle θ reflects the local orientation of the lipid.

3.10.1 List of Options

Option	Remarks	Option	Remarks
`-stype`	Common	`-sfile`	Common
`-tintype`	Common	`-tin`	Common
`-flist_traj`	Common	`-fhead`	Common
`-selX`	Common	`-mode`	Common
`-cindex`	Mode specific	`-prep_only`	Common

- **-selX <selection of atoms>**
  - Default: N/A
  - Example: `-sel0 name C32 H2X H2Y`

  Specify the carbon atom and its bonded hydrogen atoms for which S_{CD} will be calculated. Use `-selX` for each carbon atom of interest.

- **-cindex <carbon indices>**
  - Default: N/A
  - Example: `-cindex 1 2 3 4 5`

  Specifies the carbon index number (counted from the base of the acyl chain) corresponding to each `-selX` entry. The number of values given must match the number of `-selX` options. The carbon indices provided here will be used for the first column (x-axis) of the output `.scd` file.

3.10.2 Output File Format

- `*.scd`
  The S_{CD} values are printed in the order of carbon atoms specified by `-sel0`, `-sel1`, $\cdots$
  - Column 1: Carbon index (as specified by `-cindex`)
  - Column 2: S_{CD}

3.10.3 Examples of Analysis

3.10.3.1 Calculation of S_{CD} in a DPPC Lipid Bilayer

```
anatra lipid_order                              \
  -stype      psf                               \
  -sfile      complex.psf                       \
  -tintype    netcdf                            \
  -flist_traj flist                             \
  -fhead      run_nc                            \
  -dt         0.1                               \
  -cindex     1 2 3 4 5                         \
  -sel0       name C23 H3R H3S and segid MEMB \
  -sel1       name C24 H4R H4S and segid MEMB \
  -sel2       name C25 H5X H5Y and segid MEMB \
  -sel3       name C26 H6X H6Y and segid MEMB \
  -sel4       name C27 H7X H7Y and segid MEMB \
  -prep_only  false
```

3.11 Distribution Function Along the Membrane Normal (z-axis) (`z_profile`, `zprof`)

The `z_profile` (`zprof`) mode calculates the distribution of molecular species along the membrane normal (i.e., the z-axis). The local density field of species a along the z-axis is defined as

$$\hat{\rho}_a(z) = \frac{1}{A_{xy}} \sum_{i=1}^{N_{\mathrm{a}}} \delta(z - z_i), \tag{3.40}$$

where A_{xy} is the cross-sectional area of the simulation box in the xy-plane, and z_i is the z-coordinate of the i-th molecule. The ensemble-averaged distribution function is defined as

$$\rho_a(z) = \langle \hat{\rho}_a(z) \rangle. \tag{3.41}$$

For comparison with experimental data, it is often useful to compute the electron density distribution $\rho_a^e(z)$. The corresponding local density field is defined as

$$\hat{\rho}_a^e(z) = \sum_{i=1}^{N_a} \sum_{\lambda=1}^{N_s} q_\lambda \delta(z - z_{i,\lambda}), \tag{3.42}$$

where q_λ is the number of electrons on the λ-th atom in molecule a, and $z_{i,\lambda}$ is its z-coordinate in the i-th molecule. The ensemble average yields the electron density distribution: $\rho_a^e(z) = \langle \hat{\rho}_a^e(z) \rangle$.

3.11.1 List of Options

Option	Remarks	Option	Remarks
`-stype`	Common	`-sfile`	Common
`-tintype`	Common	`-tin`	Common
`-flist_traj`	Common	`-fhead`	Common
`-sel0`	Common	`-sel1`	Common
`-mode0`	Common	`-mode1`	Common
`-target_selid`	Mode specific	`-center_selid`	Mode specific
`-denstype`	Mode specific	`-symmetrize`	Mode specific
`-prep_only`	Common	–	–

- **-sel0 <selection>**
- **-sel1 <selection>**
  - Default: N/A
  - Example: `-sel0 resname ETOH -sel1 segid MEMB`

  Select the molecules for which the distribution function is to be calculated. To define the membrane center as $z = 0$, membrane molecules must also be selected.

- **-target_selid <selid>**
  - Default: N/A
  - Example: `-target_selid 0`

  Specifies the selid (defined via `-selX`) corresponding to the molecules for which the distribution is to be calculated.

- **-center_selid <selid>**
  - Default: N/A
  - Example: `-center_selid 1`

Specifies the selid (defined via `-selX`) corresponding to the membrane molecules whose center-of-mass is used to define $z = 0$.

- **-symmetrize <true or false>**
  - Default: `false`
  - Example: `-symmetrize false`

  Specifies whether to symmetrize the distribution by averaging the data for $z > 0$ and $z < 0$.

- **-denstype <number or electron>**
  - Default: `number`
  - Example: `-denstype number`

  Specifies the type of density to compute:
  `number` for number density, or `electron` for electron density.

3.11.2 Output File Format

- `*.zprof`

  The distribution function of the molecules corresponding to the `-target_selid` will be output in the following format:
  - Column 1: z-coordinate
  - Column 2: Density value

3.11.3 Examples of Analysis

3.11.3.1 Number Density Profile of Water in a Lipid Membrane System

```
#!/usr/bin/env bash

anatra z_profile                \
  -stype      psf               \
  -sfile      complex.psf       \
  -tintype    netcdf            \
  -flist_traj flist             \
  -fhead      run               \
  -sel0       water             \
  -sel1       segid MEMB and noh \
  -mode0      residue           \
```

(continues on next page)

(continued from previous page)

```
-mode1      whole            \
-target_selid 0              \
-center_selid 1              \
-denstype   number           \
-prep_only  false
```

3.12 Molecular Orientation Along a Specific Direction (`z_orient`, `zori`)

The `z_orient` (`zori`) mode computes the orientation distribution of molecules relative to a reference direction, typically the z-axis.
To define the orientation of a molecule, two parts of the molecule are selected: the *bottom* and the *head*. The centers of mass of these two parts are denoted by $\mathbf{R}_\mathrm{B}$ and $\mathbf{R}_\mathrm{H}$, respectively. The molecular axis $\mathbf{u}$ is then defined as

$$\mathbf{u} = \mathbf{R}_\mathrm{H} - \mathbf{R}_\mathrm{B}. \tag{3.43}$$

Let θ be the angle between the z-axis (unit vector $\mathbf{e}_z$) and the molecular axis (unit vector $\mathbf{e}_\mathrm{u}$). Then

$$\cos\theta = \mathbf{e}_z \cdot \mathbf{e}_\mathrm{u}. \tag{3.44}$$

The orientation distribution function is defined by

$$P(\chi) = C\,\langle \delta(\chi - \cos\theta)\rangle\,, \tag{3.45}$$

where C is a normalization constant determined such that

$$\int_0^\pi d\theta\, \sin\theta\, P(\chi) = 1. \tag{3.46}$$

By changing variables from θ to $\chi = \cos\theta$, the left hand side of the above equation becomes

$$\begin{aligned}\int_0^\pi d\theta\, \sin\theta\, P(\chi) &= \int_0^1 d\chi\, P(\chi) \\ &= CN\end{aligned} \tag{3.47}$$

where N is the number of targer molecules. Thus, C is set to $C = 1/N$. The `z_orient` mode calculates $P(\chi) = P(\cos\theta)$.

Additionally, the reference z-axis does not have to be fixed in the laboratory frame; it can also be defined by the orientation of a particular molecule. If four points $\mathbf{x}_{\mathrm{B}_0}, \mathbf{x}_{\mathrm{H}_0}, \mathbf{x}_{\mathrm{B}_1}, \mathbf{x}_{\mathrm{H}_1}$ are chosen from this reference molecule, the two vectors:

$$\mathbf{v}_0 = \mathbf{x}_{\mathrm{H}_0} - \mathbf{x}_{\mathrm{B}_0}, \quad \mathbf{v}_1 = \mathbf{x}_{\mathrm{H}_1} - \mathbf{x}_{\mathrm{B}_1} \tag{3.48}$$

can be used to define the z-axis unit vector via the cross product:

$$\mathbf{e}_z = \frac{\mathbf{v}_1 \times \mathbf{v}_2}{|\mathbf{v}_1 \times \mathbf{v}_2|}. \tag{3.49}$$

The orientation distribution $P(\cos\theta)$ is then computed based on the angle between this $\mathbf{e}_z$ and the molecular axis $\mathbf{u}$.

3.12.1 List of Options

Option	Remarks	Option	Remarks
`-stype`	Common	`-sfile`	Common
`-tintype`	Common	`-tin`	Common
`-flist_traj`	Common	`-fhead`	Common
`-sel`x	Common	`-mode`x	Common
`-zdef_type`	Mode specific	`-vecX_bottom_selid`	Mode specific
`-vecX_head_selid`	Mode specific	`-bottom_selid`	Mode specific
`-head_selid`	Mode specific	`-judgeup`	Mode specific
`-nmolup`	Mode specific	`-center_selid`	Mode specific
`-dx`	Mode specific	`-dt`	Mode specific
`-prep_only`	Common	–	–

- **-sel**x **<selection>** (up to 7 selections)
  - Default: N/A
  - Example: `-sel0 name P -sel1 name N`

  Specify the atomic groups used to define the bottom and head parts of the molecule. If using a reference molecular orientation to define the z-axis (`-zdef_type outerprod`), additional selections for $\mathbf{v}_0$ and $\mathbf{v}_1$ must also be provided. If you use `-judgeup`, selection of reference molecules to determine direction is also necessary.

- **-head_selid**, **-bottom_selid**
  - Default: N/A
  - Example: `-head_selid 0 -bottom_selid 1`

  Specify which selections correspond to the head and bottom of the molecule used to compute the orientation vector $\mathbf{u}$.

- **-zdef_type <coord or outerprod>**,
  - Default: N/A
  - Example: `-zdef_type coord`

  Required for specifying the definition of the z-direction.
  - `coord`: Use laboratory frame z-axis
  - `outerprod`: Use cross product of vectors $\mathbf{v}_0$ and $\mathbf{v}_1$ to define z-axis

  If `outerprod`, the following options smust be set.
  - `-vec0_bottom_selid` $\rightarrow \mathbf{x}_{\mathrm{B}_0}$

  - `-vec0_head_selid` $\rightarrow \mathbf{x}_{\mathrm{H}_0}$
  - `-vec1_bottom_selid` $\rightarrow \mathbf{x}_{\mathrm{B}_1}$
  - `-vec1_head_selid` $\rightarrow \mathbf{x}_{\mathrm{H}_1}$

- **-judgeup <nmolup coord or none>**
  - Default: `coord`
  - Example: `-judgeup coord`
- **-nmolup <# of target molecules in upper leaflet>**
  - Default: 0
  - Example: `-nmolup 100`
- **-center_selid <selid>**
  - Default: N/A
  - Example: `-center_selid 2`

This option specifies the orientation of $\mathbf{e}_z$ when taking the laboratory coordinate system′s z-axis as the reference. The unit vector pointing in the positive z-axis direction is denoted as $\mathbf{e}'_z$.

- In the case of `-judgeup nmolup`: Let N_{up} be the number specified by the `-nmolup` option. Then, the first N_{up} molecules are assigned $\mathbf{e}_z = \mathbf{e}'_z$ for calculating $\cos\theta$, and the remaining molecules are assigned $\mathbf{e}_z = -\mathbf{e}'_z$ for calculating $\cos\theta$.
- In the case of `-judgeup coord`: Let z_{c} be the z-coordinate of the center of mass of the molecule specified by `-center_selid`, and z_{B} and z_{H} be the z-coordinates of the centers of mass of the regions specified by `-bottom_selid` and `-head_selid`, respectively. Then,

$$\mathbf{e}_z = \begin{cases} \mathbf{e}'_z & \dfrac{z_{\mathrm{H}} + z_{\mathrm{B}}}{2} > 0 \\ -\mathbf{e}'_z & \dfrac{z_{\mathrm{H}} + z_{\mathrm{B}}}{2} \leq 0 \end{cases} \tag{3.50}$$

- **-dx <grid width for $\cos\theta$>**
  - Default: `0.1`
  - Example: `-dx 0.1`

  Grid width $\Delta\chi$ used for computing $P(\cos\theta)$.

- **-dt <time step>**
  - Default: `1.0`

  - Example: `-dt 0.1`

  Time increment for the output time series data.

3.12.2 Output File Format

- `*.costheta`
  Time series data of $\cos\theta$:
  - Column 1: Time
  - Column 2: $\cos\theta$ of molecule 1
  - Column 3: $\cos\theta$ of molecule 2
    …
- `*.distr`
  Probability distribution $P(\cos\theta)$:
  - Column 1: $\cos\theta$
  - Column 2: $P(\cos\theta)$

3.12.3 Examples of Analysis

3.12.3.1 Orientation Distribution of Lipids in a Bilayer

```
#!/usr/bin/env bash

anatra z_orient                    \
  -stype       psf                 \
  -sfile       complex.psf         \
  -tintype     nc                  \
  -flist_traj  flist               \
  -fhead       run_nc              \
  -judgeup     coord               \
  -dx          0.1                 \
  -dt          0.1                 \
  -sel0        name P              \
  -sel1        name N              \
  -sel2        segid MEMB and noh  \
  -mode0       residue             \
  -mode1       residue             \
  -mode2       whole               \
  -bottom_selid 0                  \
```

(continues on next page)

(continued from previous page)

```
-head_selid   1                 \
-center_selid 2                 \
-prep_only   false
```

3.13 Free Volume Profile (free_volume)

`free_volume` calculates the profile of the free volume fraction along a user-specified coordinate axis. The free volume is defined as the region where no atoms specified by `target_selid` exist. The atomic size is estimated from the Lennard-Jones parameters, and the free volume fraction is evaluated as the probability that randomly inserted particles do not overlap with any target atoms.

At present, only the z direction (`-coord_type z`) is supported. Before the analysis, the system is translated so that the center of the atoms specified by `center_selid` is aligned, and then the free volume fraction is calculated as a function of the z coordinate.

3.13.1 List of Options

Option	Remark	Option	Remark
`-stype`	Common	`-sfile`	Common
`-tintype`	Common	`-tin`	Common
`-flist_traj`	Common	`-fhead`	Common
`-fprmtop`	Mode specific	`-xsta`	Mode specific
`-dx`	Mode specific	`-ngrid`	Mode specific
`-nins`	Mode specific	`-sel0`	Common
`-sel1`	Common	`-target_selid`	Mode specific
`-center_selid`	Mode specific	`-coord_type`	Mode specific
`-prep_only`	Common	–	–

- **-fprmtop <Amber parm7 file>**
  - Example: `-fprmtop system.parm7`

  Specifies the Amber parm7 file from which the Lennard-Jones parameters are obtained. In the current implementation, only the parm7 format is supported.

- **-xsta <initial coordinate (Å)>**
  - Example: `-xsta -30.0`

  Specifies the starting position of the coordinate range for the profile calculation.

- **-dx <grid interval (Å)>**
  - Example: `-dx 1.0`

  Specifies the interval between neighboring grid points where the free volume fraction is evaluated.

- **-ngrid <number of grids>**
  - Example: `-ngrid 60`

  Specifies the number of grid points.

- **-nins <number of particle insertions>**
  - Example: `-nins 1000`

  Specifies the number of random particle insertions performed at each grid point. A larger value improves the statistical accuracy at the cost of increased computational time.

- **-target_selid <selection ID of target atoms>**
  - Default: `0`
  - Example: `-target_selid 0`

  Specifies the selection ID of the target atoms used to define the free volume. The region that does not overlap with these atoms is regarded as the free volume.

- **-center_selid <selection ID for centering>**
  - Default: `1`
  - Example: `-center_selid 1`

  Specifies the selection ID used to determine the center of the system. The coordinates are shifted based on this group before the analysis.

- **-coord_type <coordinate axis>**
  - Default: `z`
  - Example: `-coord_type z`

  Specifies the coordinate axis along which the free volume profile is calculated. In the current implementation, only `z` is supported.

3.13.2 Format of Output Files

- `*.profile`

  The free volume fraction profile along the specified coordinate axis is output.

  - Column 1: Coordinate
  - Column 2: Free volume fraction

3.13.3 Example

3.13.3.1 Free volume profile in a lipid membrane system

```
#!/usr/bin/env bash

anatra free_volume               \
  -stype          psf            \
  -sfile          complex.psf    \
  -tintype        nc             \
  -flist_traj     flist          \
  -fhead          out            \
  -fprmtop        complex.prmtop \
  -xsta           0.0            \
  -dx             1.0            \
  -ngrid          60             \
  -nins           1000           \
  -sel0           all            \
  -sel1           resname POPC   \
  -target_selid   0              \
  -center_selid   1              \
  -coord_type     z              \
  -prep_only      false
```

CHAPTER

FOUR

USAGE OF FANATRA

4.1 Introduction

fanatra stands for "**Fortran ANATRA**" and, as its name suggests, is a collection of analysis programs written in Fortran. This chapter describes how to use **fanatra**. Since **fanatra** performs analyses without invoking VMD, it can also be used on computing systems where the use of VMD on compute nodes is difficult. A list of available analysis modes can be displayed by executing `fanatra -h`.

```
$ fanatra -h
Available analysis in Fortran ANATRA
histogram               (hist)   : Histogram Analysis
potential_of_mean_force (pmf)    : Free-energy Profile Analysis
restricted_radial       (rrpmf)  : Restricted Radial Analysis
extract_frame           (extf)   : Extract Frames Using CV
random_frame            (randf)  : Randomly Extract Frames
state_define            (sdef)   : Define States From Time-series Data
moving_average          (movave) : Moving Average Analysis
average_function        (avef)   : Function Average Analysis
bootstrap_prepper       (boot)   : Bootstrap Sample Generator
return_prob             (rp)     : Returning Probability Analysis
rprate                  (rprate) : Rate Constant Analysis based on RP theory
iepdyn                  (iepdyn) : Integral-equation Formalism of Population Dynamics
```

Among the above, entries such as `histogram` are analysis modes, and their abbreviated names (e.g., `hist`) can also be used.

While **anatra** performs analyses by specifying parameters directly on the command line, **fanatra** uses a separate parameter file. For example,

```
$ fanatra hist control_file
```

The parameter file must be written in the Fortran namelist format.

In addition, the format and contents of the parameter file for each analysis mode can be displayed together with a brief explanation and a sample input by executing `fanatra <analysis_mode> -h`. Therefore, the basic usage of each analysis mode

can be understood from the help message. An example for `histogram` is shown below.

```
$ fanatra histogram -h

================================================

              Histogram Analysis

================================================
&input_param
 fcv      = "cvdata1" "cvdata2" ! Time-series CV data
 flist_cv = "list" ! CV-file list (Either fcv or flist_cv should be specified)
/
&output_param
 fhead = "run" ! header of output filename
/
&option_param
  dx       = 0.1d0  ! grid spacing
  xsta     = 0.0d0  ! minimum value of the coordinate
  nx       = 100    ! number of grids
  ncol     = 1      ! number of data column
/


:::
1st column: time (arbitrary), Nth column (N>1): data
example)

0.00   0.02120
0.01   0.03140
0.02   0.22310
....   .......
```

4.2 Common Parameters

Most parameter files in **fanatra** are based on the Fortran namelist format. Parameters are specified in the **input_param**, **output_param**, and **option_param** sections.

The **input_param** and **output_param** sections, which are common to all analysis modes, are described below.

4.2.1 &input_param

This namelist specifies parameters related to input files.

Parameter	Function	Example
ftraj	Specify input trajectory file(s) (multiple files allowed)	ftraj = “inp1.dcd”, “inp2.dcd”
fcv	Specify input time-series data file(s) (multiple files allowed)	fcv = “cv1.txt”, “cv2.txt”
fweight	Specify statistical weight file(s) corresponding to the input time-series data (multiple files allowed)	fweight = “weight1.txt”, “weight2.txt”
flist_traj[1]	Specify a file containing a list of input trajectory files	flist_traj = “list.txt”
flist_cv[2]	Specify a file containing a list of input time-series data files	flist_cv = “list.txt”
flist_weigh	Specify a file containing a list of statistical weight files	flist_weight = “list.txt”

[1] If both `ftraj` and `flist_traj` are specified, the contents of `flist_traj` take precedence.
[2] If both `fcv` and `flist_cv` are specified, the contents of `flist_cv` take precedence.
[3] If both `fweight` and `flist_weight` are specified, the contents of `flist_weight` take precedence.

- **`ftraj`**
  - Default: N/A
  - Example: `ftraj = "inp1.dcd", "inp2.dcd"`

  Specifies the input trajectory file(s). As shown in the example, multiple files can be specified. If the number of files is large, they can instead be specified using `flist_traj`.

- **`fcv`**
  - Default: N/A
  - Example: `fcv = "cv1.txt", "cv2.txt"`

  Specifies the input time-series data file(s). As shown in the example, multiple files can be specified. If the number of files is large, they can instead be specified using `flist_cv`.

- **`fweight`**
  - Default: N/A
  - Example: `fweight = "weight1.txt", "weight2.txt"`

  Specifies the input statistical weight file(s).

  Statistical weight files provide a statistical weight for each step of the corresponding time-series data. Statistical weights obtained from enhanced sampling methods or the Multistate Bennett Acceptance Ratio (MBAR) method can be used. The total weight is automatically normalized to unity within the program.

As shown in the example, multiple files can be specified. If the number of files is large, they can instead be specified using `flist_weight`.

- **`flist_traj`**
  - Default: N/A
  - Example: `flist_traj = "list.txt"`

  Specifies a file containing a list of input trajectory files. The contents specified by `flist_traj` take precedence over those specified by `ftraj`.

  The list file must contain one input trajectory filename per line, as shown below.

```
inp1.dcd
inp2.dcd
inp3.dcd
inp4.dcd
```

- **`flist_cv`**
  - Default: N/A
  - Example: `flist_cv = "list.txt"`

  Specifies a file containing a list of input time-series data files. The contents specified by `flist_cv` take precedence over those specified by `fcv`.

  The list file must contain one input time-series data filename per line, as shown below.

```
inp1.txt
inp2.txt
inp3.txt
inp4.txt
```

- **`flist_weight`**
  - Default: N/A
  - Example: `flist_weight = "list.txt"`

  Specifies a file containing a list of input statistical weight files. The contents specified by `flist_weight` take precedence over those specified by `fweight`.

  The list file must contain one input statistical weight filename per line, as shown below.

```
inp1.txt
inp2.txt
inp3.txt
inp4.txt
```

4.2.2 Format of Time-series Data

The format of the time-series data is as follows.

- 1st column: Time axis
- 2nd column: First time-series data
- 3rd column: Second time-series data

The first column is ignored in all analysis modes, and the second column is treated as the first data column.

4.2.3 Format of Statistical Weight Data

The format of the statistical weight data is as follows.

- 1st column: Time axis
- 2nd column: Statistical weight

The first column is ignored in all analysis modes, and only the second column is used.

The number of rows in the statistical weight data file must be identical to that of the corresponding time-series data file.

4.2.4 &output_param

This namelist specifies parameters related to output files.

Parameter	Function	Example
fhead	Specify the basename of the output file	fhead = “run”
file_extension[1]	Specify the extension of the output file	file_extension = “dat”

[1] This parameter is enabled only for a limited number of analysis modes.

- **`fhead`**
  - Default: `run`
  - Example: `fhead = "run"`

Specifies the basename of the output file.

For example, if `fhead = "run"` is specified, the analysis mode `histogram`, which is described next, outputs a file named `run.dat` by default.

- **`file_extension`**
  - Default: `dat`
  - Example: `file_extension = "dat"`

  Specifies the extension of the output file. However, this parameter is enabled only for a limited number of analysis modes, such as `histogram`.

4.3 Histogram (histogram, hist)

In this analysis mode, the distribution function $P(x)$ is calculated from a given time-series data set $\xi(t)$.

$$P(x) = \frac{1}{N} \langle \delta(x - \xi) \rangle \tag{4.1}$$

where N is the number of data points. The above expression satisfies the normalization condition

$$\int_{-\infty}^{\infty} dx P(x) = 1 \tag{4.2}$$

4.3.1 List of Parameters

Namelist	Parameter	Remarks
`&input_param`	`flist_cv`	–
`&input_param`	`fcv`	–
`&output_param`	`fhead`	–
`&output_param`	`file_extension`	–
`&option_param`	`dx`	Grid spacing of the distribution function
`&option_param`	`xsta`	Starting position of the grid
`&option_param`	`nx`	Number of grid points
`&option_param`	`ncol`	Number of data columns per file

4.3.2 &option_param

- **dx**
  - Default: 0.1
  - Example: `dx = 0.1`

  Specifies the grid spacing Δx used to calculate the distribution function.

- **xsta**
  - Default: 0.0
  - Example: `xsta = 0.0`

  Specifies the starting position of the grid, x_0. If the position of the i-th grid point is denoted by x_i,

$$x_i = x_0 + (i - 1)\,\Delta x \tag{4.3}$$

  holds.

- **ncol**
  - Default: 1
  - Example: `ncol = 10`

  Specifies the number of data columns contained in each time-series data file.

4.3.3 Format of Output File

- `*.dat` (the extension can be changed by `file_extension`)
  - 1st column: x coordinate
  - 2nd column: Distribution function $P\,(x)$

4.3.4 Example

The following example calculates the distribution function $P\,(x)$ in the range $5 \leq x \leq 10$ from a time-series data file `inp.ts` containing three data columns.

```
#!/usr/bin/env bash

cat << EOF > run.inp
&input_param
fcv = "inp.ts"
```

(continues on next page)

(continued from previous page)

```
/
&output_param
fhead = "run"
/
&option_param
dx   = 0.1d0
xsta = 5.0d0
nx   = 50
ncol = 3
/
EOF

fanatra hist run.inp
```

4.4 Potential of Mean Force (potential_of_mean_force, pmf)

In this analysis mode, the free-energy profile $W(\zeta)$ along the reaction coordinate ζ is calculated from the time-series data of the reaction coordinate, $\xi(t)$. The function $W(\zeta)$ is also referred to as the **potential of mean force (PMF)**.

4.4.1 Theoretical Background

Let $\mathbf{X}$ denote the configuration of the system and $U(\mathbf{X})$ its potential energy. The configurational integral Z is given by

$$Z = \int d\mathbf{X} \exp\left(-\beta U(\mathbf{X})\right) \tag{4.4}$$

where β is the inverse temperature.

Similarly, the partition function corresponding to the state in which the system is trapped within the infinitesimal region $\zeta - \zeta + \Delta\zeta$ in the reaction coordinate space ($\zeta(\mathbf{X})$) is

$$Z(\zeta) = \Delta\zeta \int d\mathbf{X}\, \delta\left(\zeta - \xi(\mathbf{X})\right) \exp\left(-\beta U(\mathbf{X})\right) \tag{4.5}$$

Defining the probability density corresponding to ζ as

$$\rho(\zeta) = \langle \delta\left(\zeta - \xi(\mathbf{X})\right) \rangle = \frac{1}{Z} \int d\mathbf{X}\, \delta\left(\zeta - \xi(\mathbf{X})\right) \exp\left(-\beta U(\mathbf{X})\right) \tag{4.6}$$

we obtain

$$\frac{Z(\zeta)}{Z} = \rho(\zeta)\, \Delta\zeta \tag{4.7}$$

Therefore, the free-energy difference $\Delta F(\zeta)$ between the unrestricted state and the state confined to the region $\zeta - \zeta + \Delta\zeta$ is expressed as

$$\Delta F(\zeta) = -\frac{1}{\beta} \log \frac{Z(\zeta)}{Z} = -\frac{1}{\beta} \log\left(\rho(\zeta)\, \Delta\zeta\right) \tag{4.8}$$

The free energy relative to a reference point $\boldsymbol{\zeta}_0$ is

$$\begin{aligned} W(\boldsymbol{\zeta}) &= \Delta F(\boldsymbol{\zeta}) - \Delta F(\boldsymbol{\zeta}_0) \\ &= -\frac{1}{\beta} \log \frac{\rho(\boldsymbol{\zeta})}{\rho(\boldsymbol{\zeta}_0)} \end{aligned} \tag{4.9}$$

This quantity W is called the **potential of mean force**. The choice of $\boldsymbol{\zeta}_0$ is arbitrary, but in most cases it is chosen such that the minimum value of $W(\boldsymbol{\zeta})$ becomes zero.

By calculating $W(\boldsymbol{\zeta})$, it becomes possible to discuss the change in thermodynamic stability as a function of $\boldsymbol{\zeta}$.

Equation (4.9) can also be rewritten by separating the $\boldsymbol{\zeta}$-dependent part and a constant C:

$$W(\boldsymbol{\zeta}) = -\frac{1}{\beta} \log \int d\mathbf{X}\, \delta(\boldsymbol{\zeta} - \boldsymbol{\xi}(\mathbf{X})) \exp(-\beta U(\mathbf{X})) + C \tag{4.10}$$

where

$$C = \frac{1}{\beta} \log \int d\mathbf{X}\, \delta(\boldsymbol{\zeta}_0 - \boldsymbol{\xi}(\mathbf{X})) \exp(-\beta U(\mathbf{X})) \tag{4.11}$$

4.4.2 List of Parameters

Namelist	Parameter	Remarks
`&input_param`	`flist_cv`	Filename containing a list of CV files
`&input_param`	`flist_weight`	Filename containing a list of weight files (optional)
`&input_param`	`fcv`	CV filename
`&input_param`	`fweight`	Weight filename (optional)
`&output_param`	`fhead`	Header of the output filename
`&option_param`	`ndim`	Number of CV dimensions
`&option_param`	`xyzcol`	Column indices of the CV data
`&option_param`	`ng3`	Number of grid points for each CV direction
`&option_param`	`del`	Grid spacing for each CV direction
`&option_param`	`origin`	Grid origin for each CV direction
`&option_param`	`temperature`	Temperature [K]
`&option_param`	`use_radial`	Whether radial CVs are included
`&option_param`	`radcol`	Column indices corresponding to radial CVs
`&option_param`	`box_system`	Simulation box size (used only when `ndim=3`)
`&option_param`	`use_spline`	Enable spline interpolation
`&option_param`	`spline_resolution`	Resolution of spline interpolation
`&option_param`	`gen_script_mpl2d`	Generate a Python script for 2D PMF visualization
`&option_param`	`skip_calc`	Skip PMF calculation
`&matplotlib_param`	`fpdf`	Output PDF filename
`&matplotlib_param`	`labels`	Axis labels and colorbar label
`&matplotlib_param`	`ranges`	Plotting range
`&matplotlib_param`	`tics`	Axis tick intervals
`&matplotlib_param`	`scales`	Unit conversion factors for each axis

4.4.3 &option_param

- **ndim**
  - Default: None
  - Example: `ndim = 2`

  Specifies the number of dimensions of the Collective Variable (CV).

  For example, `ndim = 2` calculates a two-dimensional PMF, while `ndim = 3` calculates a three-dimensional PMF.

- **xyzcol**
  - Default: `1`
  - Example: `xyzcol = 1 2`

  Specifies the column indices corresponding to each CV in the CV data file.

  The first column is ignored because it is treated as the time or step index.

  For example, if the data are stored as

```
time  cv1  cv2  cv3
```

  then

```
xyzcol = 1 2
```

  calculates a two-dimensional PMF using `cv1` and `cv2`.

- **ng3**
  - Default: `1 1 1`
  - Example: `ng3 = 10 20`

  Specifies the number of grid points for each CV direction.

  When `ndim = N`, the grid numbers for each dimension should be specified sequentially in `ng3`.

- **del**
  - Default: `1.0 1.0 1.0`
  - Example: `del = 0.1d0 0.2d0`

  Specifies the grid spacing for each CV direction.

  For the i-th CV, if the grid spacing is denoted by Δ_i, the distance between adjacent grid points is Δ_i.

- **origin**
  - Default: `0.0 0.0 0.0`
  - Example: `origin = 0.0d0 1.0d0`

  Specifies the grid origin for each CV direction.

  For the k-th CV, let the origin be $x_k^{(0)}$ and the grid spacing be Δ_k. Then the position of the i-th grid point is given by

$$x_k^{(i)} = x_k^{(0)} + (i-1)\Delta_k \tag{4.12}$$

- **temperature**
  - Default: `300.0d0`
  - Example: `temperature = 298.0d0`

  Specifies the temperature T [K] used for the PMF calculation.

- **use_radial**
  - Default: `.false.`
  - Example: `use_radial = .true.`

  Specifies whether the CV contains a radial component.

  If `.true.`, the distribution function for the CV component x_k specified by `radcol` is calculated using the bin volume

$$4\pi x_k^2 \Delta_k^2 \tag{4.13}$$

- **radcol**
  - Default: N/A
  - Example: `radcol = 1`

  Specifies which CV corresponds to the radial component.

  This parameter is effective only when `use_radial = .true.`.

- **box_system**
  - Default: `0.0 0.0 0.0`
  - Example: `box_system = 50.0d0 50.0d0 50.0d0`

Specifies the simulation box size.

This parameter is used only when `ndim = 3`. The box lengths in each direction should be specified in order.

- **`use_spline`**
  - Default: `.false.`
  - Example: `use_spline = .true.`

  Specifies whether spline interpolation is performed after the PMF calculation.

  If `.true.`, spline interpolation is applied; otherwise, the original grid is used.

- **`spline_resolution`**
  - Default: `4`
  - Example: `spline_resolution = 8`

  Specifies the resolution used for spline interpolation.

  Larger values generate a finer interpolation grid.

- **`gen_script_mpl2d`**
  - Default: `.false.`
  - Example: `gen_script_mpl2d = .true.`

  Specifies whether a Python script for visualizing a two-dimensional PMF is generated.

  Currently, this option is supported only when `ndim = 2`.

- **`skip_calc`**
  - Default: `.false.`
  - Example: `skip_calc = .true.`

  Specifies whether the PMF calculation is skipped.

  If `.true.`, the PMF calculation is omitted and only auxiliary files are generated. If `.false.`, the PMF calculation is performed normally.

  This option is convenient when recreating the Matplotlib (Python) script after modifying its settings.

4.4.4 &matplotlib_param

Options for generating a Python (Matplotlib) script for visualizing a two-dimensional PMF.

These options are enabled when `gen_script_mpl2d = .true.` is specified in `&option_param`.

- **`fpdf`**
  - Default: None
  - Example: `fpdf = "pmf.pdf"`

  Specifies the name of the output PDF file.

- **`labels`**
  - Default: None
  - Example: `labels = "$x$" "$y$" "PMF"`

  Specifies the axis labels displayed in the plot.

  When `ndim = 2`,

```
labels = "xlabel" "ylabel" "zlabel"
```

  specifies the labels for the x-axis, the y-axis, and the color bar corresponding to the PMF values, respectively.

  LaTeX-style mathematical expressions can be used.

- **`ranges`**
  - Default: None
  - Example: `ranges = -10.0 10.0 -5.0 5.0 0.0 10.0`

  Specifies the plotting range.

  When `ndim = 2`,

```
ranges = xmin xmax ymin ymax zmin zmax
```

  should be specified in this order.

  Here,

  - `xmin`, `xmax` : Display range of the x-axis
  - `ymin`, `ymax` : Display range of the y-axis
  - `zmin`, `zmax` : Display range of the PMF values

- **tics**
  - Default: None
  - Example: `tics = 1.0 1.0 0.5`

  Specifies the intervals of the axis ticks.

  When `ndim = 2`,

```
tics = xtic ytic ztic
```

  should be specified in this order.

  Here,

  - `xtic` : Tick interval of the x-axis
  - `ytic` : Tick interval of the y-axis
  - `ztic` : Tick interval of the color bar corresponding to the PMF values

---

- **scales**
  - Default: `1.0 1.0 1.0`
  - Example: `scales = 10.0 10.0 0.239006`

  Specifies the unit conversion factors for the axes.

  When `ndim = 2`,

```
scales = xscale yscale zscale
```

  should be specified in this order.

  The loaded data values are multiplied by these factors when they are plotted.

  For example,

```
scales = 10.0 10.0 1.0
```

  displays the values of the x- and y-axes multiplied by 10.

4.4.5 Format of Output Files

- `*_original.gnplt`

  The PMF ($W(\boldsymbol{\zeta})$) is written to this file.

  - 1st column: Coordinate
  - 2nd column: PMF

- `*_g_original.gnplt`

  The distribution function ($\rho(\boldsymbol{\zeta}) / \rho_0(\boldsymbol{\zeta}_0)$) is written to this file.

  - 1st column: Coordinate
  - 2nd column: Distribution function

When `use_spline = .true.`, the spline-interpolated PMF and distribution function are written to `*_spline.gnplt` and `*_g_spline.gnplt`, respectively.

4.4.6 Example

The following example calculates the PMF ($W(x)$) with a grid spacing of $\Delta x = 0.1$ over the range $5 \leq x \leq 10$ from a time-series data file `inp.ts` containing a single data column.

```
#!/usr/bin/env bash

cat << EOF > run.inp
&input_param
 fcv     = "inp.ts"
/
&output_param
 fhead = "out"
/
&option_param
  ndim        = 1
  xyzcol      = 1
  ng3         = 50
  del         = 0.1
  origin      = 5.0
  temperature = 300.0d0
  use_spline  = .false.
  use_radial  = .false.
/
EOF
```

(continues on next page)

(continued from previous page)

```
fanatra potential_of_mean_force run.inp
```

4.5 Restricted Radial Potential of Mean Force (restricted_radial, rrpmf)

This analysis mode calculates the free energy profile along the radial direction under a condition defined by one or more reaction coordinates (collective variables).

Using this profile, the free energy change associated with the binding of a host molecule and a guest molecule (binding free energy) can also be evaluated.

4.5.1 Theoretical Background

Let $\mathbf{X}$ denote the configuration of the system. The intermolecular distance between the host and guest molecules and the reaction coordinates defining the binding state are denoted by $d(\mathbf{X})$ and $\boldsymbol{\xi}(\mathbf{X})$, respectively.

The potential of mean force along the intermolecular distance is defined as

$$w(r) = -\frac{1}{\beta}\log\left[\int d\mathbf{X}\,\frac{\delta(r - d(\mathbf{X}))}{4\pi r^2}\exp\left(-\beta U(\mathbf{X})\right)\right] + C \tag{4.14}$$

Furthermore, let $\boldsymbol{\Upsilon}$ be the region in the reaction coordinate space corresponding to the bound state, and introduce the characteristic function $\Theta(\boldsymbol{\xi})$ defined as

$$\Theta_{\mathrm{B}}(\boldsymbol{\xi}) = \begin{cases} 1 & \boldsymbol{\xi} \in \boldsymbol{\Upsilon} \\ 0 & \boldsymbol{\xi} \notin \boldsymbol{\Upsilon} \end{cases} \tag{4.15}$$

Using this function, the PMF for configurations belonging to the bound state is expressed as

$$w_{\mathrm{B}}(r) = -\frac{1}{\beta}\log\left[\int d\mathbf{X}\,\frac{\delta(r - d(\mathbf{X}))}{4\pi r^2}\Theta_{\mathrm{B}}(\boldsymbol{\xi}(\mathbf{X}))\exp\left(-\beta U(\mathbf{X})\right)\right] + C \tag{4.16}$$

The constant C appearing in both $w(r)$ and $w_{\mathrm{B}}(r)$ is taken to be the same value, and is chosen such that the minimum value of $w_{\mathrm{B}}(r)$ becomes zero.

Since $w(r)$ becomes constant when r is sufficiently large, let this constant be Δw, and define the region satisfying $w(r) = \Delta w$ as $r_0 \leq r \leq r_1$. The choice of r_0 and r_1 is arbitrary as long as this condition is satisfied. This point will be discussed later.

The probability ratio between the unbound state U, where the two molecules are separated by $r_0 \leq r \leq r_1$, and the bound state B is

$$\frac{\displaystyle\int_{\boldsymbol{\Upsilon}} dr\,\exp\left(-\beta w_{\mathrm{B}}(r)\right)}{\displaystyle\int_{r_0}^{r_1} dr\,\exp\left(-\beta w(r)\right)} \tag{4.17}$$

Therefore, the free energy change ΔG_{PMF} associated with the transition from U to B is

$$\Delta G_{\mathrm{PMF}} = -\frac{1}{\beta} \log \frac{\displaystyle\int_{\boldsymbol{\Upsilon}} dr\, 4\pi r^2 \exp\left(-\beta w_{\mathrm{B}}\left(r\right)\right)}{\displaystyle\int_{r_0}^{r_1} dr\, 4\pi r^2 \exp\left(-\beta w\left(r\right)\right)} \tag{4.18}$$

Since $w\left(r\right) = \Delta w$ within the range $r_0 \leq r \leq r_1$, this equation can be rewritten as

$$\Delta G_{\mathrm{PMF}} = -\Delta w - \frac{1}{\beta} \log \frac{V_{\mathrm{B}}}{V_{\mathrm{U}}} \tag{4.19}$$

where

$$V_{\mathrm{B}} = \int_{\boldsymbol{\Upsilon}} dr\, 4\pi r^2 \exp\left(-\beta w_{\mathrm{B}}\left(r\right)\right) \tag{4.20}$$

$$V_{\mathrm{U}} = \int_{r_0}^{r_1} dr\, 4\pi r^2 \tag{4.21}$$

The second term in Eq. (4.19) represents the free energy change associated with the transition of the guest molecule from a concentration of $1/V_{\mathrm{U}}$ to a concentration of $1/V_{\mathrm{B}}$.

The experimentally measured binding free energy is the free energy change associated with binding a guest molecule at the standard-state concentration (typically $c^\circ = 1\ \mathrm{mol\ L^{-1}}$), and is therefore different from ΔG_{PMF}.

Let V° be the volume corresponding to a concentration of c° for a single guest molecule. Then $V^\circ = 1661\ \mathrm{Å}^3$.

The free energy change $\Delta G^\circ_{\mathrm{corr}}$ associated with the transition from concentration $c^\circ = 1/V^\circ$ to $1/V_{\mathrm{U}}$ is

$$\Delta G^\circ_{\mathrm{corr}} = -\frac{1}{\beta} \log \frac{V_{\mathrm{U}}}{V^\circ} \tag{4.22}$$

Thus, by decomposing the transition from $1/V^\circ$ to $1/V_{\mathrm{B}}$ into the two steps,

$$1/V^\circ \to 1/V_{\mathrm{U}} \tag{4.23}$$

and

$$1/V_{\mathrm{U}} \to 1/V_{\mathrm{B}} \tag{4.24}$$

we obtain

$$\Delta G^\circ = \Delta G_{\mathrm{PMF}} + \Delta G^\circ_{\mathrm{corr}} \tag{4.25}$$

Substituting Eqs. (4.19) and (4.22) into the above equation yields

$$\Delta G^\circ = -\Delta w - \frac{1}{\beta} \log \frac{V_{\mathrm{B}}}{V^\circ} \tag{4.26}$$

Since this expression does not contain V_{U}, the value of ΔG° is independent of the choice of r_0 and r_1.

4.5.2 Parameter List

Namelist	Parameter	Description
&input_param	flist_cv	File containing the list of CV files
&input_param	flist_weight	File containing the list of weight files (optional)
&input_param	fcv	CV filename
&input_param	fweight	Weight filename
&output_param	fhead	Header of the output filename
&option_param	calcfe	Whether to calculate the standard free energy
&option_param	ndim	Number of CV dimensions
&option_param	only_r	Whether the intermolecular distance is the only CV
&option_param	temperature	Temperature [K]
&option_param	dr	Distance grid spacing
&option_param	ngrid	Number of distance grids
&option_param	nsta	Starting frame number for the analysis
&option_param	use_bootstrap	Whether to perform bootstrap analysis
&option_param	vol0	Standard-state volume
&option_param	urange	Distance range of the unbound state
&option_param	bound_range	Distance range of the bound state
&bootopt_param	iseed	Random number seed
&bootopt_param	nsample	Number of samples drawn in each trial
&bootopt_param	duplicate	Whether duplicate sampling is allowed
&bootopt_param	ntrial	Number of bootstrap trials

4.5.3 Format of the Input File (Time-Series Data)

- 1st column: Time
- 2nd to $(N+1)$-th columns: CV values
- Last column: Intermolecular distance

The first column is ignored, and the second column is treated as the first data column.

If the intermolecular distance is the only CV, i.e.,

- 1st column: Time
- 2nd column: Intermolecular distance

then simply specify

```
only_r = .true.
```

in &option_param.

4.5.4 &option_param

- **calcfe**
  - Default: `.true.`
  - Example: `calcfe = .true.`

  Specifies whether the standard binding free energy should be calculated.

  If `.true.`, the standard binding free energy is calculated. If `.false.`, only the PMF profile is generated.

- **ndim**
  - Default: None
  - Example: `ndim = 1`

  Specifies the number of CV dimensions.

  Currently, both one-dimensional and multidimensional PMF calculations are supported.

- **only_r**
  - Default: `.false.`
  - Example: `only_r = .false.`

  Set this option to `.true.` when the intermolecular distance is the only CV and no additional CVs are present.

- **temperature**
  - Default: `298.0`
  - Example: `temperature = 300.0`

  Specifies the temperature T [K] used for the free energy calculation.

- **dr**
  - Default: None
  - Example: `dr = 0.1d0`

  Specifies the grid spacing along the distance coordinate.

  This value is used as the grid interval Δr.

- **ngrid**
  - Default: None

  - Example: `ngrid = 1000`

  Specifies the number of grid points along the distance coordinate.

  The PMF is evaluated on `ngrid` grid points.

- **nsta**
  - Default: `1`
  - Example: `nsta = 100`

  Specifies the frame number from which the analysis starts.

  Frames before `nsta` are excluded from the analysis.

- **use_bootstrap**
  - Default: `.false.`
  - Example: `use_bootstrap = .true.`

  Specifies whether bootstrap error analysis is performed.

  If `.true.`, bootstrap analysis is carried out according to the settings in `&bootopt_param`.

- **vol0**
  - Default: `1661.0d0`
  - Example: `vol0 = 1661.0d0`

  Specifies the standard-state volume V°.

  The default value corresponds to the volume for a concentration of 1 mol/L in Å units (1661 Å^3).

- **urange**
  - Default: None
  - Example: `urange = 25.0 30.0`

  Specifies the distance range regarded as the unbound state.

  The first value represents the lower bound, and the second value represents the upper bound.

- **bound_range**
  - Default: None
  - Example:

```
react_range = 0.0 10.0
              5.0 20.0
```

Specifies the CV range corresponding to the bound state.

For each line, the first value represents the lower bound and the second value represents the upper bound.

By specifying `ndim` ranges, a multidimensional bound region can be defined.

4.5.5 &bootopt_param

This namelist controls the settings for bootstrap error analysis.

It is enabled only when `use_bootstrap = .true.`.

- **iseed**
  - Default: Automatically generated
  - Example: `iseed = 3141592`

  Specifies the random seed used for bootstrap analysis.

  If `iseed <= 0`, the program automatically generates a seed.

- **nsample**
  - Default: None
  - Example: `nsample = 1000`

  Specifies the number of samples drawn in each bootstrap trial.

- **duplicate**
  - Default: `.true.`
  - Example: `duplicate = .false.`

  Specifies whether duplicate sampling is allowed during bootstrap sampling.

- **ntrial**
  - Default: None
  - Example: `ntrial = 100`

  Specifies the number of bootstrap trials.

  Increasing the number of trials improves the accuracy of the statistical error estimate, but also increases the computational cost.

4.5.6 Output File Format

- `*.rd`
  - 1st column: Intermolecular distance r
  - 2nd column: Conventional PMF along r
  - 3rd column: PMF corresponding to the bound state

4.5.7 Example

The following example calculates the binding free energy from a time-series data file `inp.ts` that contains one CV and the intermolecular distance in the last column.

The range $18 \leq r/\text{Å} \leq 20$ is treated as the unbound state, and the bound state is defined by the CV range $5.0 \leq \zeta \leq 10.0$.

```
#!/usr/bin/env bash

cat << EOF > run.inp
&input_param
 fcv      = "inp.ts"
/
&output_param
 fhead         = "out"
/
&option_param
 calcfe        = .true.
 only_r        = .false.
 ndim          =  1
 temperature   =  300.0
 dr            =  0.1d0
 ngrid         =  1000
 nsta          =  1
 use_bootstrap = .false.
 vol0          = 1661.0d0
 urange        =  18.0  20.0
 bound_range   =  5.0  10.0
/
EOF

fanatra restricted_radial run.inp
```

After execution, the results are printed near the end of the log file.

```
... Thermodynamic analysis ...
vb      (A^3)      =      21.5444908
wu      (kcal/mol) =      -1.3486059
wb      (kcal/mol) =       0.0000000
dw      (kcal/mol) =      -1.3486059
dGv     (kcal/mol) =       2.5903536
dGzero  (kcal/mol) =       3.9389595
Keq     (M^-1)     =   1.3506025E-03
```

4.6 Frame Extraction (extract_frame, extf)

In this analysis mode, snapshots (frames) that satisfy specified CV ranges are extracted.

4.6.1 Parameter List

Namelist	Parameter	Description
&input_param	flist_traj	Filename containing the list of trajectory files
&input_param	flist_cv	Filename containing the list of CV files
&input_param	ftraj	Trajectory filename
&input_param	fcv	CV filename
&output_param	fhead	Header of the output filename
&option_param	dt	Time interval of the trajectory
&option_param	ndim	Number of CV dimensions
&option_param	extract_range	CV range for frame extraction

4.6.2 &option_param

- **ndim**
  - Default: None
  - Example: `ndim = 2`

  Specifies the number of dimensions of the time-series CV data.

  The ranges specified in `extract_range` must correspond to this dimensionality.

- **extract_range**
  - Default: None

- Example:

```
extract_range = -5.0d0 5.0d0
                 1.0d0 0.0d0
```

Specifies the CV region from which frames are extracted.

For each CV, a pair of lower and upper bounds should be provided.

In the above example, frames satisfying

- CV1: $-5.0 \leq \mathrm{CV}_1 \leq 5.0$
- CV2: $0.0 \leq \mathrm{CV}_2 \leq 1.0$

are extracted.

By specifying `ndim` ranges, multidimensional extraction conditions can be defined.

4.6.3 Output File Format

The output trajectory file has the same format as the input trajectory file (dcd, xtc, or nc (netcdf)).

4.6.4 Example

The following example extracts frames satisfying the condition $5 \leq \zeta \leq 10$ from an input trajectory, using a time-series CV data file `inp.ts` containing a single CV.

```
#!/usr/bin/env bash

cat << EOF > run.inp
&input_param
 ftraj    = "input.dcd"
 fcv      = "inp.ts"
/
&output_param
 fhead         = "out"
/
&option_param
 ndim          =  1
 extract_range =  5.0  10.0
/
EOF

fanatra extract_frame run.inp
```

After execution, the extracted trajectory is written to `out.dcd`.

4.7 Random Frame Extraction (random_frame, randf)

In this analysis mode, snapshots (frames) are randomly extracted from a trajectory.

4.7.1 Parameter List

Namelist	Parameter	Description
`&input_param`	`ftraj`	Input trajectory filename
`&input_param`	`flist_traj`	Filename containing the list of trajectory files
`&output_param`	`fhead`	Header of the output filename
`&option_param`	`nsample`	Number of randomly extracted frames
`&option_param`	`iseed`	Random seed
`&option_param`	`output_trajtype`	Output trajectory format
`&option_param`	`duplicate`	Whether duplicate sampling is allowed
`&option_param`	`out_rst7`	Whether to output AMBER rst7 files
`&option_param`	`shuffle`	Whether to shuffle the extracted frames

4.7.2 &option_param

- **`nsample`**
  - Default: `100`
  - Example: `nsample = 100`

  Specifies the number of frames to be randomly extracted.

- **`iseed`**
  - Default: `-1` (automatically generated)
  - Example: `iseed = 3141592`

  Specifies the random seed used for frame extraction.

  If `iseed <= 0`, the program automatically generates a seed.

- **`output_trajtype`**
  - Default: `dcd`

  - Example: `output_trajtype = "dcd"`

  Specifies the output trajectory format.

  Currently, the following formats are supported:

  - `dcd`
  - `xtc`
  - `netcdf`

- **`duplicate`**
  - Default: `.false.`
  - Example: `duplicate = .true.`

  Specifies whether the same frame can be extracted multiple times.

- **`out_rst7`**
  - Default: `.false.`
  - Example: `out_rst7 = .true.`

  Specifies whether each extracted frame should also be output in AMBER rst7 format.

- **`shuffle`**
  - Default: `.false.`
  - Example: `shuffle = .true.`

  Specifies whether the extracted frames should be shuffled before output.

  If `.true.`, all extracted frames are written in a random order.

4.7.3 Output File Format

The output trajectory file has the same format as the input trajectory file (dcd, xtc, or nc (netcdf)).

4.7.4 Example

The following example extracts 100 random frames from the trajectory file `input.dcd`. In addition to the trajectory, the structure of each extracted frame is also written in `rst7` (`inpcrd`) format.

```
#!/usr/bin/env bash

cat << EOF > run.inp
&input_param
 ftraj    = "input.dcd"
/
&output_param
 fhead         = "out"
/
&option_param
 nsample = 100
 output_trajtype = "dcd"
 duplicate       = .false.
 out_rst7        = .true.
/
EOF

fanatra random_frame run.inp
```

After execution, `out.dcd` and `outXXXXX.inpcrd` (`XXXXX = 00001, ..., 00100`) are generated.

4.8 Moving Average (moving_average, movave)

In this analysis mode, one-dimensional data (e.g., distribution functions) are smoothed using a moving average.

4.8.1 Parameter List

Namelist	Parameter	Description
`&input_param`	`fcv`	Function data file
`&input_param`	`flist_cv`	File containing a list of function data files
`&output_param`	`fhead`	Output file header
`&option_param`	`dx`	Grid spacing
`&option_param`	`xsta`	Starting position of the grid
`&option_param`	`xsep`	Boundary position between regions
`&option_param`	`nregion`	Number of regions
`&option_param`	`npoint`	Number of points used for moving average in each region
`&option_param`	`include_zero`	Whether to preserve the value at the origin

4.8.2 &option_param

- **dx**
  - Default: None
  - Example: `dx = 0.1d0`

  Specifies the grid spacing of the function data.

- **xsta**
  - Default: `0.0d0`
  - Example: `xsta = 0.0d0`

  Specifies the starting position of the grid.

  If the position of the i-th grid point is denoted by x_i,

$$x_i = x_0 + (i-1)\Delta x \tag{4.27}$$

  where x_0 is the starting position.

- **nregion**
  - Default: `1`
  - Example: `nregion = 2`

  Specifies the number of regions to which different moving-average conditions are applied.

  The number of values given to `npoint` must match this number of regions.

- **xsep**
  - Default: `0.0d0`
  - Example: `xsep = 1.0d0`

  Specifies the boundary positions between regions with different moving-average widths.

  This parameter is not used when `nregion = 1`, in which case `xsep = 0.0d0` should be specified.

  For example, when dividing the domain into N regions, $N-1$ boundary values must be specified.

- **npoint**
  - Default: None
  - Example: `npoint = 3 5`

Specifies the number of data points used for the moving average in each region.

An odd number must be specified.

In the above example,

- Region 1: 3-point moving average
- Region 2: 5-point moving average

are applied.

- `include_zero`
  - Default: `.true.`
  - Example: `include_zero = .false.`

  Specifies whether to preserve the value at the origin after smoothing.

  If `.true.`, the value at the origin is kept unchanged from the input data.

4.8.3 Output File Format

- `*XXXX.dat`
  - Column 1: x coordinate
  - Column 2: Function value

  A smoothed data file is generated for each input file specified by `flist_cv` or related options.

4.8.4 Example

The following example smooths a one-dimensional function $f(x)$ stored in `input.dat` (grid origin `0`, grid spacing `0.1`).

The domain is divided into two regions, $x \leq 1.0$ and $x > 1.0$, and 3-point and 5-point moving averages are applied to the first and second regions, respectively.

```
#!/usr/bin/env bash

cat << EOF > run.inp
&input_param
 fcv      = "input.dat"
/
&output_param
 fhead = "out"
/
&option_param
```

(continues on next page)

(continued from previous page)

```
  dx           = 0.1d0
  xsta         = 0.0d0
  xsep         = 1.0d0
  nregion      = 2
  npoint       = 3 5
  include_zero = .true.
/
EOF

fanatra moving_average run.inp
```

Executing this command generates the smoothed function data file `out0001.dat`.

4.9 Average Function (average_function, avef)

In this analysis mode, one-dimensional functions stored in multiple input files are averaged. For example, let $f_i(x)$ be the function stored in the i-th file and let N be the total number of files. The averaged function $f_{\mathrm{ave}}(x)$ is given by

$$f_{\mathrm{ave}}(x) = \frac{1}{N} \sum_{i=1}^{N} f_i(x), \tag{4.28}$$

4.9.1 Parameter List

Namelist	Parameter	Description
&input_param	flist_cv	File containing a list of function data files
&output_param	fhead	Output file header
&option_param	use_bootstrap	Enable bootstrap analysis
&option_param	xsta	Starting position of the grid
&option_param	dx	Grid spacing
&bootopt_param	duplicate	Whether duplicate sampling is allowed
&bootopt_param	iseed	Random seed
&bootopt_param	nsample	Number of samples drawn in each trial
&bootopt_param	ntrial	Number of bootstrap trials

4.9.2 &option_param

- **use_bootstrap**
  - Default: `.false.`
  - Example: `use_bootstrap = .true.`

  Specifies whether to perform error estimation using the bootstrap method.

  If `.true.`, bootstrap analysis is carried out according to the settings in `&bootopt_param`.

- **xsta**
  - Default: `0.0d0`
  - Example: `xsta = 0.0d0`

  Specifies the starting position of the grid.

  If the position of the i-th grid point is denoted by x_i,

$$x_i = x_0 + (i-1)\Delta x \tag{4.29}$$

  where x_0 is the grid origin.

- **dx**
  - Default: `0.1d0`
  - Example: `dx = 0.1d0`

  Specifies the grid spacing of the function data.

4.9.3 &bootopt_param

This section defines the settings for bootstrap error analysis.

It is effective only when `use_bootstrap = .true.`.

- **iseed**
  - Default: Automatically generated
  - Example: `iseed = 3141592`

  Specifies the random seed used for bootstrap analysis.

  If `iseed <= 0`, the program automatically generates a seed.

- **nsample**

  - Default: None
  - Example: `nsample = 1000`

  Specifies the number of samples drawn in each bootstrap trial.

- **`duplicate`**
  - Default: `.true.`
  - Example: `duplicate = .false.`

  Specifies whether duplicate sampling is allowed during bootstrap sampling.

- **`ntrial`**
  - Default: None
  - Example: `ntrial = 100`

  Specifies the number of bootstrap trials.

  Increasing the number of trials improves the accuracy of the estimated statistical error, but also increases the computational cost.

4.9.4 Output File Format

- `*.ave`
  - Column 1: x coordinate
  - Column 2: Function value
  - Column 3: Standard error (when `use_bootstrap = .true.`)

4.9.5 Example

Suppose that one-dimensional functions $f(x)$ (grid origin `0`, grid spacing `0.1`) are stored in multiple files, and the list of these files is stored in `list`. The following example calculates the average function and its standard error using the bootstrap method.

```
#!/usr/bin/env bash

cat << EOF > run.inp
&input_param
 flist_cv = "list"
/
&output_param
```

(continues on next page)

(continued from previous page)

```
 fhead         = "out"
/
&option_param
 use_bootstrap =  .false.
 xsta          =  0.0d0
 dx            =  0.1d0
/

&bootopt_param
 duplicate     = .true.
 iseed         =  3141592
 nsample       =  1000
 ntrial        =  100
/
EOF

fanatra average_function run.inp
```

Executing this command generates the averaged function data file `out.ave`.

CHAPTER

FIVE

INTEGRAL-EQUATION FORMALISM OF POPULATION DYNAMICS (IEPDYN)

The Integral-Equation Formalism of Population DYNamics (IEPDYN) is a theoretical framework for describing transitions between discrete states.

A key feature of this method is that it represents state dynamics as a continuous-time process. Unlike conventional approaches such as the Markov State Model (MSM), IEPDYN does not require the introduction of a discrete time scale (lag time), enabling a description of dynamics without the lag-time dependence.

The IEPDYN requires the calculation of time-correlation functions associated with local state transitions. Since these quantities contain only information on local state-to-state transitions, they can be evaluated from short MD simulations. Accordingly, IEPDYN enables the prediction of long-time population dynamics using information obtained from short-time MD simulations.

In ANATRA, the IEPDYN method is implemented as one of the analysis modes of `fanatra`.

5.1 Theoretical Background

5.1.1 Theoretical Framework of IEPDYN

Let $\mathbf{\Gamma}$ denote the set of atomic coordinates and momenta (a phase point) of the system, and divide the corresponding phase space into multiple structural states.

If $\mathcal{L}$ is the Liouville operator of the system, the exact time evolution of the probability density of state j, denoted by $\hat{f}_j\left(\mathbf{\Gamma}, t\right)$, is given by

$$\frac{\partial}{\partial t}\hat{f}_j\left(\mathbf{\Gamma}, t\right) = -\mathcal{L}\hat{f}_j\left(\mathbf{\Gamma}, t\right) - \sum_{i \in \mathcal{N}_j} S_{ij}\left(\mathbf{\Gamma}\right)\hat{f}_j\left(\mathbf{\Gamma}, t\right) + \sum_{k \in \mathcal{N}_j} S_{jk}\left(\mathbf{\Gamma}\right)\hat{f}_k\left(\mathbf{\Gamma}, t\right). \tag{5.1}$$

Here, $\mathcal{N}_j$ is the set of states adjacent to state j, and S_{ij} is called the **reaction sink function**. It is defined such that

$$S_{ij}\left(\mathbf{\Gamma}\right)\hat{f}_j\left(\mathbf{\Gamma}, t\right) \tag{5.2}$$

represents the probability that the system transitions from state j to state i during the time interval from t to $t + dt$.

The probability that the system is in state j at time t, denoted by $P_j\left(t\right)$, is obtained by integrating $\hat{f}_j\left(\mathbf{\Gamma}, t\right)$ over the phase space.

From the formal solution of the above equation, the following approximate expression for $P_j\,(t)$ is obtained:

$$P_j\,(t) = P_j^0\,(t) + \sum_{k \in \mathcal{N}_j} \int_0^t d\tau\, M_{jk}\,(t-\tau)\, Q_{jk}\,(\tau)\,, \tag{5.3}$$

where

- $P_j^0\,(t)$ is the probability that a system initially in state j remains in state j without making any transition to neighboring states up to time t.
- $Q_{jk}\,(t)$ is the probability that the system transitions from state k to state j during the time interval from t to $t + dt$.
- $M_{jk}\,(t)$ is the probability that a system that entered state j from state k at time zero remains in state j without any further transitions to neighboring states up to time t.

Since $P_j^0(t)$ and $M_{jk}(t)$ involve only a single transition event, they can be evaluated from short MD simulations.

Furthermore, $Q_{jk}\,(t)$ can also be represented by an integral equation whose kernel decays on a short time scale:

$$Q_{ij}\,(t) = R_{ij}\,(t) + \sum_{k \in \mathcal{N}_j} \int_0^t d\tau\, K_{ijk}\,(t-\tau)\, Q_{jk}\,(\tau)\,, \tag{5.4}$$

where

- $R_{ij}\,(t)$ is the probability that a system initially in state j transitions to state i during the time interval from t to $t + dt$.
- $K_{ijk}\,(t)$ is the probability that a system that transitioned from state k to state j during the interval $0 \sim dt$ subsequently transitions to state i during the time interval from t to $t + dt$.

Like $P_j^0\,(t)$ and $M_{jk}\,(t)$, both $R_{ij}\,(t)$ and $K_{ijk}\,(t)$ can be computed from short MD simulations. Moreover, $P_j^0\,(t)$ and $M_{jk}\,(t)$ can be constructed from $R_{ij}\,(t)$ and $K_{ijk}\,(t)$. Therefore, by performing short MD simulations and evaluating $R_{ij}\,(t)$ and $K_{ijk}\,(t)$, one can predict long-time population dynamics $P_j\,(t)$.

In the IEPDYN framework, the initial population of each state, w_j, must be specified. If the system is assumed to be in equilibrium at the initial time, let $\mathcal{M}_{\mathrm{I}}$ be the set of states that have nonzero initial populations and let P_j^{eq} be the equilibrium population of state j. Thus,

$$\forall j \in \mathcal{M}_{\mathrm{I}},\; w_j = \frac{P_j^{\mathrm{eq}}}{\sum_{k \in \mathcal{M}_{\mathrm{I}}} P_k^{\mathrm{eq}}} \tag{5.5}$$

The equilibrium populations P_j^{eq} may be obtained from free energy calculations such as umbrella sampling. Alternatively, as described later, they can also be evaluated analytically within the IEPDYN framework.

5.1.2 Introducing Boundary Conditions

In the IEPDYN framework, reflecting and absorbing boundary conditions can be introduced into the coarse-grained reaction dynamics. These boundary conditions are imposed directly on the state-to-state transition probabilities, $Q_{ij}(t)$, and are particularly useful for analyzing molecular binding processes based on the recurrence probability (RP) theory.

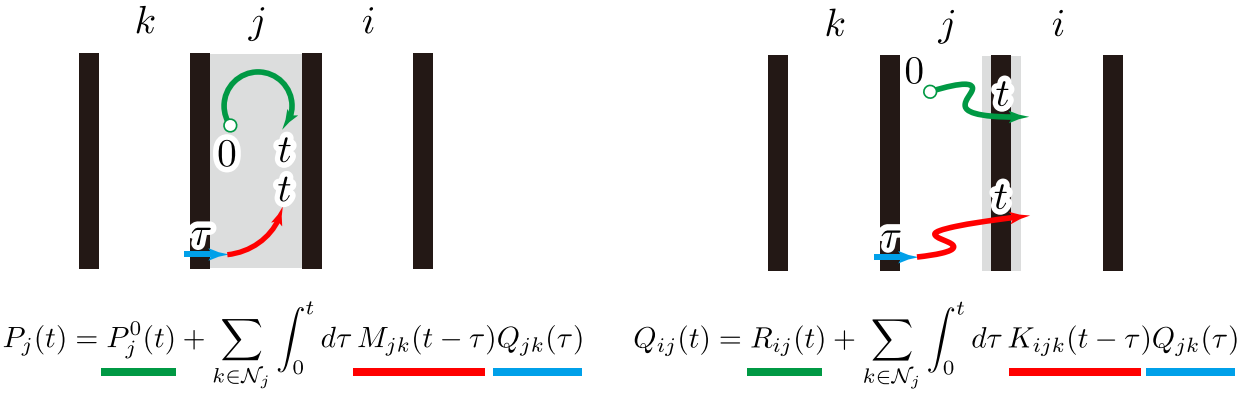

Fig. 5.1: Schematic illustration of the IEPDYN.

For the **reflecting boundary condition**, any population entering a reflecting state is immediately redistributed to neighboring states without remaining in that state. Let $\mathcal{M}_{\text{reflect}}$ denote the set of reflecting states. The reflecting boundary condition is expressed as

$$\forall i \in \mathcal{M}_{\text{reflect}},\ \forall j \in \mathcal{N}_i,$$

$$Q_{ji}(t) = R_{ij}(t) + \sum_{k\in\mathcal{N}_j} \int_0^t d\tau\, K_{ijk}(t-\tau)\, Q_{jk}(\tau), \tag{5.6}$$

$$Q_{ij}(t) = 0, \tag{5.7}$$

which represents that a population flowing from state j into a reflecting state $i \in \mathcal{M}_{\text{reflect}}$ is instantaneously reflected back to state j.

In contrast, under the **absorbing boundary condition**, a population that reaches an absorbing state never transitions to any other state thereafter. Let $\mathcal{M}_{\text{absorb}}$ denote the set of absorbing states. The absorbing boundary condition is given by

$$\forall i \in \mathcal{M}_{\text{absorb}},\ \forall j \in \mathcal{N}_i, \quad Q_{ji}(t) = 0, \tag{5.8}$$

which means that no population can flow out of an absorbing state. Once the population reaches an absorbing state, it is treated as having reached the final state.

In this way, the IEPDYN framework can treat both reflecting and absorbing boundaries in a unified manner by directly imposing boundary conditions on the state-to-state transition probabilities.

5.1.3 Incorporating Returning Probability (RP) Theory

The Returning probability (RP) theory is a rigorous framework of diffusion-influenced reactions. This theory characterizes molecular binding processes in terms of the thermodynamic and kinetic properties of the **reactive state** existing during binding processes. In the IEPDYN framework, the dynamical quantities required by the RP theory can be efficiently evaluated from short MD simulations.

In the RP theory, the association rate constant, k_{on}, is given by

$$k_{\mathrm{on}} = k_{\mathrm{ins}} K^* \left(1 + k_{\mathrm{ins}} \int_0^\infty dt\, P_{\mathrm{RET}}\left(t\right) \right)^{-1}, \tag{5.9}$$

where K^* is the equilibrium constant between the reactive and unbound states, k_{ins} is the transition rate constant from the reactive state to the bound state, and $P_{\mathrm{RET}}\left(t\right)$ is the conditional probability (returning probability) that a system initially in the reactive state is still in the reactive state at time t.

In the IEPDYN framework, let $\mathcal{M}_{\mathrm{R}}$ denote the set of reactive states, and set the initial-state set as

$$\mathcal{M}_{\mathrm{I}} = \mathcal{M}_{\mathrm{R}}. \tag{5.10}$$

By introducing appropriate boundary conditions, both k_{ins} and $P_{\mathrm{RET}}(t)$ can be evaluated from the state populations $P_j(t)$.

For the calculation of k_{ins}, an absorbing boundary condition is introduced to define the completion of the binding process, while a reflecting boundary condition is imposed on the neighboring states located on the unbound side of the reactive state. Under these conditions,

$$\frac{1}{k_{\mathrm{ins}}} = \int_0^\infty dt \sum_{j \notin \mathcal{M}_{\mathrm{absorb}}} P_j\left(t\right), \tag{5.11}$$

holds. The right-hand side corresponds to the **Mean First Passage Time (MFPT)** from the reactive state to the absorbing state, and its inverse gives k_{ins}.

On the other hand, for the calculation of $P_{\mathrm{RET}}(t)$, the neighboring states located on the bound side of the reactive state are treated as reflecting boundaries. In this case,

$$P_{\mathrm{RET}}\left(t\right) = \sum_{j \in \mathcal{M}_{\mathrm{R}}} P_j\left(t\right), \tag{5.12}$$

gives the returning probability.

Thus, by appropriately introducing boundary conditions, the IEPDYN framework enables efficient evaluation of both k_{ins} and $P_{\mathrm{RET}}(t)$ from short MD simulations. Furthermore, by combining these quantities with the equilibrium constant K^*, the association rate constant k_{on} for long-time-scale molecular binding processes can be determined.

5.1.4 Analytical Estimation of the Time Integral of $P_j\left(t\right)$

Kinetic parameters, including k_{ins} in the RP theory, are expressed in terms of the time integral of the state population,

$$\tau_j = \int_0^\infty dt\, P_j(t). \tag{5.13}$$

By introducing absorbing boundary conditions or states corresponding to infinite separation, $P_j(t)$ converges to zero as $t \to \infty$. A straightforward approach is to solve the integral equations for the time evolution of $P_j(t)$ followed by its numerical integration. Instead of the numerical integration, the IEPDYN enables us to directly evaluate τ_j through the Laplace transform.

The Laplace transform of a function $f(t)$ is defined as

$$\widetilde{f}(s) = \int_0^\infty dt\, e^{-st} f(t). \tag{5.14}$$

Applying the Laplace transform to the integral equation for the state population yields

$$\widetilde{P}_j(s) = \widetilde{P}_j^0(s) + \sum_{k \in \mathcal{N}_j} \widetilde{M}_{jk}(s)\, \widetilde{Q}_{jk}(s). \tag{5.15}$$

Taking the limit $s \to 0$, we obtain

$$\tau_j = \widetilde{P}_j^0(0) + \sum_{k \in \mathcal{N}_j} \widetilde{M}_{jk}(0)\, \widetilde{Q}_{jk}(0). \tag{5.16}$$

Therefore, the evaluation of τ_j reduces to the calculation of $\widetilde{Q}_{jk}(0)$. To this end, we introduce

$$\mathcal{K}_{ij,j'k}(t) = \delta_{jj'} K_{ijk}(t). \tag{5.17}$$

The integral equation for $Q_{ij}(t)$ can then be written in Laplace space as

$$\widetilde{Q}_{ij}(s) = \widetilde{R}_{ij}(s) + \sum_{j',k} \widetilde{\mathcal{K}}_{ij,j'k}(s)\, \widetilde{Q}_{j'k}(s). \tag{5.18}$$

By introducing a composite index λ for the state pair (ij), the above equation can be expressed in matrix form as

$$\widetilde{\mathbf{Q}}(s) = \widetilde{\mathbf{R}}(s) + \widetilde{\boldsymbol{\mathcal{K}}}(s)\widetilde{\mathbf{Q}}(s). \tag{5.19}$$

Taking the limit $s \to 0$ gives

$$\widetilde{\mathbf{Q}}(0) = \left[\mathbf{I} - \widetilde{\boldsymbol{\mathcal{K}}}(0)\right]^{-1} \widetilde{\mathbf{R}}(0), \tag{5.20}$$

where $\mathbf{I}$ is the identity matrix and

$$\left(\mathbf{I} - \widetilde{\boldsymbol{\mathcal{K}}}(0)\right)_{\lambda\mu} = \delta_{\lambda\mu} - \int_0^\infty dt\, \mathcal{K}_{\lambda\mu}(t). \tag{5.21}$$

Thus, the time-integrated transition probability, $\widetilde{\mathbf{Q}}(0)$, can be obtained directly by matrix operations, and τ_j can be evaluated analytically. This approach eliminates the need to explicitly compute the long-time evolution of $P_j(t)$ and enables efficient evaluation of kinetic quantities such as the MFPT.

5.1.5 Analytical Evaluation of the Equilibrium Population $P_j(\infty)$

For systems in which the total population is conserved, the equilibrium population of state j,

$$P_j^{\mathrm{eq}} \equiv P_j(\infty), \tag{5.22}$$

can be evaluated without numerically integrating the time evolution of $P_j(t)$.

First, the state-transition probability $Q_{ij}(t)$ is decomposed into its equilibrium and time-dependent components as

$$Q_{ij}(t) = Q_{ij}^{\mathrm{eq}} + \delta Q_{ij}(t). \tag{5.23}$$

where

$$Q_{ij}^{\mathrm{eq}} \equiv Q_{ij}(\infty) \tag{5.24}$$

represents the equilibrium population flux from state j to state i.

Substituting this decomposition into the IEPDYN integral equation and taking the limit $t \to \infty$, we obtain

$$P_j^{\mathrm{eq}} = \lim_{t\to\infty} \left[\sum_{k\in\mathcal{N}_j} \int_0^t d\tau\, M_{jk}(t-\tau)\delta Q_{jk}(\tau) \right] + \sum_{k\in\mathcal{N}_j} I_{jk} Q_{jk}^{\mathrm{eq}}, \tag{5.25}$$

where I_{jk} is defined as

$$I_{jk} = \int_0^\infty dt\, M_{jk}(t). \tag{5.26}$$

Since both $M_{jk}(t)$ and $\delta Q_{jk}(t)$ vanish in the limit $t \to \infty$, the first term disappears, yielding

$$P_j^{\mathrm{eq}} = \sum_{k\in\mathcal{N}_j} I_{jk} Q_{jk}^{\mathrm{eq}}. \tag{5.27}$$

Therefore, once the equilibrium transition fluxes Q_{jk}^{eq} are determined, the equilibrium populations of all states can be calculated directly.

Furthermore, the integral equation can be written in matrix form as

$$\boldsymbol{\delta}\mathbf{Q}(t) = \mathbf{R}(t) + \int_0^t d\tau\, \boldsymbol{\mathcal{K}}(t-\tau)\boldsymbol{\delta}\mathbf{Q}(\tau) - \mathbf{Q}^{\mathrm{eq}} + \mathbf{J}\mathbf{Q}^{\mathrm{eq}}, \tag{5.28}$$

where

$$J_{\lambda\mu} = \int_0^\infty dt\, \mathcal{K}_{\lambda\mu}(t). \tag{5.29}$$

In the limit $t \to \infty$, the time-dependent terms vanish, leading to

$$\mathbf{J}\mathbf{Q}^{\mathrm{eq}} = \mathbf{Q}^{\mathrm{eq}}. \tag{5.30}$$

Thus, $\mathbf{Q}^{\mathrm{eq}}$ is given by the eigenvector of the matrix $\mathbf{J}$ corresponding to the eigenvalue of unity. By solving this eigenvalue problem, the equilibrium transition fluxes can be obtained, and the equilibrium populations are then evaluated from

$$P_j^{\mathrm{eq}} = \sum_{k\in\mathcal{N}_j} I_{jk} Q_{jk}^{\mathrm{eq}}. \tag{5.31}$$

The resulting equilibrium populations are also used to determine the initial statistical weights in the IEPDYN formalism:

$$\forall j \in \mathcal{M}_{\mathrm{I}}, \quad w_j = \frac{P_j^{\mathrm{eq}}}{\sum_{k\in\mathcal{M}_{\mathrm{I}}} P_k^{\mathrm{eq}}}. \tag{5.32}$$

In binding and dissociation processes, a population-conserving system can be constructed by introducing reflecting boundary conditions at the outermost states. This enables the present method to efficiently evaluate the equilibrium populations required for IEPDYN calculations.

5.2 Parameter List

Namelist	Parameter	Description
&input_param	fcv	Time-series CV data file
&input_param	flist_cv	List of CV data files
&output_param	fhead	Output file header
&option_param	input_type	Input file format
&option_param	output_histogram	Output restart file
&option_param	use_perturbed_traj	Whether the data are generated from MD simulations under a bias potential
&option_param	f_unperturbed_id	State ID that remains unperturbed in each dataset
&option_param	use_dissociate_state	Whether to define dissociated states
&option_param	use_reflection_state	Whether to define reflecting states
&option_param	use_product_state	Whether to define product states
&option_param	calc_steady	Whether to calculate the steady-state distribution
&option_param	calc_Pint	Whether to analytically evaluate the time integral
&option_param	extrapolate	Evaluate time evolution based on the integral equations
&option_param	nstate	Number of states
&option_param	ndim	Number of CV dimensions
&option_param	nmol	Number of target molecules
&option_param	dt	Time grid spacing
&option_param	t_sparse	Coarse time grid spacing for TCF calculations
&option_param	t_range	Time range for calculating the K, M, R, and P0 functions
&option_param	t_extend	Output time range for the P and Q functions
&option_param	dt_tcfout	Output time interval for the P and Q functions
&option_param	initial_state_ids	Initial state IDs
&option_param	reflection_state_ids	Reflecting state IDs
&option_param	product_state_ids	Product state IDs
&state	state definition	State boundaries and initial populations for each state

5.3 &option_param

- **input_type**
  - Default: "TIMESERIES"
  - Example: input_type = "TIMESERIES"

  Specifies the format of the input files.

  Available formats are:

  - TIMESERIES : ANATRA standard time-series data

  - HISTOGRAM : Histogram restart files (*.krhist) generated with output_histogram = .true.

- **output_histogram**
  - Default: .false.
  - Example: output_histogram = .true.

  Specifies whether histogram-format restart files (*.krhist) are generated from the calculation results.

- **use_perturbed_traj**
  - Default: .false.
  - Example: use_perturbed_traj = .true.

  Specifies whether the input time-series data were obtained from MD simulations under a bias potential. The bias potential should not perturb the dynamics within a particular state, such as a flat-bottom potential. If .true. is specified, f_unperturbed_id must also be provided.

- **f_unperturbed_id**
  - Default: N/A
  - Example: f_unperturbed_id = "funp.list"

  This parameter is required when use_perturbed_traj = .true.. It specifies a list file that describes, for each time-series dataset,

  1. which state's dynamics are not perturbed by the bias potential, and
  2. whether the dataset is used for calculating $R_{ij}(t)$.

  The entries should correspond to the order of files specified by flist_cv or fcv.

  Each line should contain:

  - Column 1: Unperturbed state ID
  - Column 2: Whether the dataset is used for calculating $R_{ij}(t)$ (1: yes, 0: no)

  For example, if the file specified by f_unperturbed_id is

```
1 1
1 1
2 0
2 0
```

  the first two datasets correspond to trajectories whose state 1 dynamics are unperturbed and are used for calculating $R_{ij}(t)$, whereas the next two datasets correspond to trajectories whose state 2 dynamics are unperturbed but are not used for calculating $R_{ij}(t)$.

- **use_dissociate_state**
  - Default: `.false.`
  - Example: `use_dissociate_state = .true.`

  Specifies whether a dissociated state is defined.

- **use_reflection_state**
  - Default: `.false.`
  - Example: `use_reflection_state = .true.`

  Specifies whether reflecting states are defined.

- **use_product_state**
  - Default: `.false.`
  - Example: `use_product_state = .true.`

  Specifies whether absorbing (product) states are defined.

- **calc_steady**
  - Default: `.false.`
  - Example: `calc_steady = .true.`

  Specifies whether the equilibrium (steady-state) populations are evaluated analytically.

- **calc_Pint**
  - Default: `.false.`
  - Example: `calc_Pint = .true.`

  Specifies whether the time integrals of the state populations are evaluated analytically.

- **extrapolate**
  - Default: `.false.`
  - Example: `extrapolate = .true.`

  Specifies whether the time evolution of the state populations $P_j(t)$ is evaluated using the integral equations.

- **nstate**
  - Default: None
  - Example: `nstate = 4`

  Specifies the number of states.

- **ndim**
  - Default: `1`
  - Example: `ndim = 2`

  Specifies the dimensionality of the Collective Variables (CVs).

- **nmol**
  - Default: `1`
  - Example: `nmol = 1`

  Specifies the number of target molecules.

- **dt**
  - Default: None
  - Example: `dt = 0.1d0`

  Specifies the time grid interval.

- **t_sparse**
  - Default: None
  - Example: `t_sparse = 0.1d0`

  Specifies the coarse time grid interval used for calculating time correlation functions (TCFs), such as $K_{ijk}(t)$.

- **t_range**
  - Default: None
  - Example: `t_range = 10.0`

  Specifies the time range over which $K_{ijk}(t)$, $M_{jk}(t)$, $R_{ij}(t)$, and $P_j^0(t)$ are calculated.

- **t_extend**
  - Default: None

  - Example: `t_extend = 100.0`

  Specifies the time range over which $P_j(t)$ and $Q_{ij}(t)$ are output.

- **`dt_tcfout`**
  - Default: None
  - Example: `dt_tcfout = 1.0`

  Specifies the output interval for $P_j(t)$ and $Q_{ij}(t)$.

- **`initial_state_ids`**
  - Default: None
  - Example: `initial_state_ids = 2 3`

  Specifies the state IDs used as initial states.

- **`reflection_state_ids`**
  - Default: None
  - Example: `reflection_state_ids = 4`

  Specifies the state IDs treated as reflecting states.

- **`product_state_ids`**
  - Default: None
  - Example: `product_state_ids = 1`

  Specifies the state IDs treated as absorbing (product) states.

5.4 &state

In the `&state` section, the CV ranges and initial populations of each state are specified line by line as

```
lower_bound(CV1)  upper_bound(CV1) lower_bound(CV2) upper_bound(CV2) ... lower_bound(CVndim)␣
↪upper_bound(CVndim) weight
```

- Column 1: Lower bound of CV1
- Column 2: Upper bound of CV1
- Column 3: Lower bound of CV2

- Column 4: Upper bound of CV2

…

- Final column: Initial population of the state

Thus, a pair of lower and upper bounds must be provided for each CV dimension specified by `ndim`.

For example, when `ndim = 1`,

```
&state
-500.0  -16.0  0.5
 -16.0  -15.0  0.5
 -15.0   -2.0  0.0
  -2.0  500.0  0.0
/
```

defines four states, where the first two states are assigned an initial population of 0.5.

5.5 Auxiliary Tools

`iepdyn` provides many configuration options. To simplify the preparation of input files, the following Python utilities are available:

```
$ANATRA_PATH/utility/iepdyn_util
```

Both of the following scripts are interactive and can be executed without any command-line arguments.

- `iepdyn_setup.py`

  A utility for generating input files for `iepdyn`.

- `cvlist_generator.py`

  A utility for generating a list of time-series data files.

  When time-series data obtained from biased MD simulations are used (`use_perturbed_traj = .true.`), this script can classify the datasets according to the directory hierarchy.

5.6 Output Files

- `*.krhist` (`output_histogram = .true.`)

  Stores the information of $K_{ijk}(t)$ and $R_{ij}(t)$ for restart calculations. Multiple `krhist` files can be loaded simultaneously during a restart calculation. The information stored in this file corresponds to the state before applying boundary conditions. Therefore, it can be reused even if the boundary conditions are changed, without recalculating the time correlation functions.

- `*.P0`
  - Column 1: Time
  - Column 2 and later: $P_j^0(t)$

  The correspondence between columns and state indices is given in the file header.
- `*.Rij`
  - Column 1: Time
  - Column 2 and later: $R_{ij}(t)$

  The correspondence between columns and $R_{ij}(t)$ is given in the file header.
- `*.Mij`
  - Column 1: Time
  - Column 2 and later: $M_{ij}(t)$

  The correspondence between columns and $M_{ij}(t)$ is given in the file header.
- `*.Kijk`
  - Column 1: Time
  - Column 2 and later: $K_{ijk}(t)$

  The correspondence between columns and $K_{ijk}(t)$ is given in the file header.
- `*.pj` (`extrapolate = .true.`)
  - Column 1: Time
  - Column 2 and later: $P_j(t)$

  The correspondence between columns and $P_j(t)$ is given in the file header.
- `*.tcf` (`extrapolate = .true.`)
  - Column 1: Time
  - Column 2 and later: $\sum_{k \in \mathcal{M}_\mathrm{I}} P_k(t)$
- `*.q` (`extrapolate = .true.`)
  - Column 1: Time
  - Column 2 and later: $Q_{ij}(t)$

  The correspondence between columns and $Q_{ij}(t)$ is given in the file header.
- `*.int` (`calc_Pint = .true.`)

  Stores

$$\tau_j = \int_0^\infty dt\, P_j\left(t\right), \tag{5.33}$$

in the following format:

```
Pint InitTotal    0.5549862E+02
Pint         1    0.5474586E+02
Pint         2    0.6403562E+00
Pint         3    0.9158297E-01
Pint         4    0.1578925E-01
Pint         5    0.5035148E-02
Pint         6    0.7000565E-02
Pint         7    0.3941343E-01
Pint         8    0.6599485E-01
```

The second column represents the state index, and the third column gives the value of τ_j. `InitTotal` denotes the sum of τ_j over all states belonging to the initial-state set $\mathcal{M}_{\mathrm{I}}$.

- `*.steady` (`calc_steady = .true.`)
  - Column 1: Time
  - Column 2: State index (j)
  - Column 3: P_j^{eq}
  - Column 4: $-k_{\mathrm{B}}T \log \dfrac{P_j^{\mathrm{eq}}}{\displaystyle\sum_{k \in \mathcal{M}_{\mathrm{I}}} P_k^{\mathrm{eq}}}$

5.7 Examples

As an example, we describe the application procedure of the IEPDYN method to a molecular binding/dissociation process characterized by the intermolecular distance r.

The states are defined as follows:

- State 1 (bound state): $0 \leq r < 2$
- State 2 (bound state): $2 \leq r < 3.5$
- State 3: $3.5 \leq r < 4$
- State 4 (including the dissociated state): $4 \leq r$

The target quantity is defined as the residence time correlation function,

$$P_{\mathrm{B}}(t) = \frac{\displaystyle\sum_{k \in \mathcal{M}_{\mathrm{I}}} P_k(t)}{\displaystyle\sum_{k \in \mathcal{M}_{\mathrm{I}}} P_k(0)}, \tag{5.34}$$

and its time integral, τ_{B}.

τ_{B} represents the lifetime of the bound state. In this example, States 1 and 2 are treated as the initial states.

5.7.1 Using Unbiased MD Trajectories (use_perturbed_traj = .false.)

5.7.1.1 Generating Restart Files (krhist)

First, a restart file (krhist) is generated for each time-series dataset. The following example reads the time-series file 0001.ts and generates krhist/0001.krhist.

Although States 1 and 2 are the bound (initial) states, their initial population values are not used when generating the krhist file. Therefore, the initial populations in the &state section may simply be set to 1.0.

```
#!/usr/bin/env bash

mkdir -p krhist

cat << EOF > run0001.inp
&input_param
 fcv = "0001.ts"
/
&output_param
 fhead = "krhist/0001"
/
&option_param
 input_type           = "TIMESERIES"
 output_histogram     = .true.
 use_perturbed_traj   = .false.
 use_dissociate_state = .true.
 use_reflection_state = .false.
 use_product_state    = .false.
 calc_steady          = .false.
 calc_Pint            = .false.
 extrapolate          = .false.
 nstate               = 4
 ndim                 = 1
 nmol                 = 1
 dt                   = 2e-05
 t_sparse             = 2e-05
 t_range              = 1.0
 initial_state_ids    = 1 2
 dissociate_state_ids = 4
/
&state
 0.0     2.0      1.0
 2.0     3.5      1.0
```

(continues on next page)

(continued from previous page)

```
 3.5     4.0      0.0
 4.0     1.0d10   0.0
/
EOF

fanatra iepdyn run0001.inp
```

5.7.1.2 Calculation of Equilibrium Populations

To compute $P_{\mathrm{B}}(t)$ using the IEPDYN method, initial population values for states 1 and 2 are required.
Therefore, by defining dissociation state 4 as a reflection state, the population is conserved among states 1–3, and the equilibrium populations (P_j^{eq}) are computed using the previously generated restart file (`krhist`).
For this purpose, the following options are set:

- `calc_Pint = .true.`
- `use_reflection_state = .true.`
- `reflection_state_ids = 4`

and additionally,

- `use_dissociate_state = .false.`

is specified.

```
#!/usr/bin/env bash

mkdir -p steady

# Create list of krhist files
ls krhist/*.krhist > krhist.list

# Create input file
cat << EOF > steady.inp
&input_param
 flist_cv = "krhist.list"
/
&output_param
 fhead = "steady/out"
/
&option_param
 input_type           = "HISTOGRAM"
 output_histogram     = .false.
```

(continues on next page)

(continued from previous page)

```
 use_perturbed_traj   = .false.
 use_dissociate_state = .false.
 use_reflection_state = .true.
 use_product_state    = .false.
 calc_steady          = .true.
 calc_Pint            = .false.
 extrapolate          = .false.
 nstate               = 4
 ndim                 = 1
 nmol                 = 1
 dt                   = 2e-05
 t_sparse             = 2e-05
 t_range              = 1.0
 initial_state_ids    = 1 2
 reflection_state_ids = 4
/
&state
 0.0     2.0      1.0
 2.0     3.5      1.0
 3.5     4.0      0.0
 4.0     1.0d10   0.0
/
EOF

fanatra iepdyn steady.inp
```

When this is executed, `steady/out.int` is obtained, and its contents are, for example, as follows:

```
   1    0.9874940E+00    0.7502598E-02
   2    0.1066799E-01    0.2706875E+01
   3    0.1489986E-02    0.3880407E+01
```

The second column corresponds to the equilibrium population values.

5.7.1.3 Calculation of $P_{\mathrm{B}}(t)$

Using the equilibrium population values obtained in the previous step, the values for states 1 and 2 are provided in the `&state` section.

Then, the following options are specified:

- `extrapolate = .true.`
- `t_extend = 10.0`

The parameter `t_extend` defines the time range over which $P_j\,(t)$ is evaluated.

In addition, the following settings are used:

- `use_dissociate_state = .true.`
- `use_reflection_state = .false.`
- `dissociate_state_ids = 4`

It should be noted that the sum of the initial equilibrium populations is automatically normalized to 1 within the program.

```
#!/usr/bin/env bash

mkdir -p extrapolate

Create list of krhist files

ls krhist/*.krhist > krhist.list

Create input file

cat << EOF > extrapolate.inp
&input_param
 flist_cv = "krhist.list"
/
&output_param
 fhead = "extrapolate/out"
/
&option_param
 input_type           = "HISTOGRAM"
 output_histogram     = .false.
 use_perturbed_traj   = .false.
 use_dissociate_state = .true.
 use_reflection_state = .false.
 use_product_state    = .false.
 calc_steady          = .false.
 calc_Pint            = .false.
 extrapolate          = .true.
 nstate               = 4
 ndim                 = 1
 nmol                 = 1
 dt                   = 2e-05
 t_sparse             = 2e-05
 t_range              = 1.0
 t_extend             = 10.0
```

(continues on next page)

(continued from previous page)

```
 dt_tcfout            = 2e-05
 initial_state_ids    = 1 2
 dissociate_state_ids = 4
/
&state
 0.0     2.0        0.9874940E+00
 2.0     3.5        0.1066799E-01
 3.5     4.0        0.0
 4.0     1.0d10     0.0
/
EOF

fanatra iepdyn extrapolate.inp
```

Running this produces the file `extrapolate/out.tcf`, which contains the time-dependent information of $P_{\mathrm{B}}(t)$.

5.7.1.4 Analytical Estimation of the Time Integral of $P_{\mathrm{B}}(t)$

Next, the time integral of $P_{\mathrm{B}}(t)$ is evaluated analytically. In this case, the following parameters in the previous input file are modified:

- `extrapolate = .false.`
- `calc_Pint = .true.`

In addition, parameters such as `t_extend` and `dt_tcfout` do not affect the result and are removed.

```
#!/usr/bin/env bash

mkdir -p Pint

# Create list of krhist files
#
ls krhist/*.krhist > krhist.list

# Create input file
#
cat << EOF > Pint.inp
&input_param
 flist_cv = "krhist.list"
/
&output_param
 fhead = "Pint/out"
```

(continues on next page)

(continued from previous page)

```
/
&option_param
 input_type           = "HISTOGRAM"
 output_histogram     = .false.
 use_perturbed_traj   = .false.
 use_dissociate_state = .true.
 use_reflection_state = .false.
 use_product_state    = .false.
 calc_steady          = .false.
 calc_Pint            = .true.
 extrapolate          = .false.
 nstate               = 4
 ndim                 = 1
 nmol                 = 1
 dt                   = 2e-05
 t_sparse             = 2e-05
 t_range              = 1.0
 initial_state_ids    = 1 2
 dissociate_state_ids = 4
/
&state
 0.0     2.0        0.9874940E+00
 2.0     3.5        0.1066799E-01
 3.5     4.0        0.0
 4.0     1.0d10     0.0
/
EOF

fanatra iepdyn Pint.inp
```

When this is executed, the file `Pint/out.int` is obtained.
The third column of the first line in this file corresponds to the time-integrated value of $P_{\mathrm{B}}(t)$.

5.7.2 Using Biased MD Trajectories (using `use_perturbed_id = .true.`)

The procedure is almost the same as in the unbiased case, but it becomes slightly more complicated because it is necessary to prepare a list specified by `f_unperturbed_id`.

5.7.2.1 Generating Restart Files (`krhist`)

Here, a `krhist` file is generated for each time-series dataset.
It is assumed that the time-series data in which the dynamics of state j are not perturbed are organized in directories such as:

```
ts/0001/run0001.dis
ts/0001/run0002.dis
...
```

Based on this directory structure, first the lists corresponding to `flist_cv` and `f_unperturbed_id` are generated using the following Python script:

```
$ $ANATRA_PATH/utility/iepdyn_util/cvlist_generator.py
```

When executed, the script prompts the user for input as follows:

```
=== File Search Utility ===

(1) Enter the root directory to search: ts
(2) Enter the target extension (e.g., txt): dis
(3) Enter the maximum search directory depth: 2

Searching files...

=== Matched Files ===

/path/to/ts/0001/run0001.dis
/path/to/ts/0001/run0002.dis
/path/to/ts/0002/run0001.dis
/path/to/ts/0002/run0002.dis
/path/to/ts/0003/run0001.dis
/path/to/ts/0003/run0002.dis
/path/to/ts/0004/run0001.dis
/path/to/ts/0004/run0002.dis

(4) Enter directory level counted from the deepest directory for grouping (default: 1): 1

=== Grouped Files ===
```

(continues on next page)

(continued from previous page)

```
--- State 1: 0001 ---
/path/to/ts/0001/run0001.dis
/path/to/ts/0001/run0002.dis

--- State 2: 0002 ---
/path/to/ts/0002/run0001.dis
/path/to/ts/0002/run0002.dis

--- State 3: 0003 ---
/path/to/ts/0003/run0001.dis
/path/to/ts/0003/run0002.dis

--- State 4: 0004 ---
/path/to/ts/0004/run0001.dis
/path/to/ts/0004/run0002.dis

Did these files come from perturbed dynamics? (y/n): y

Available states:
1 : 0001
2 : 0002
3 : 0003
4 : 0004

Enter state numbers used for Rij calculation (space-separated, empty input allowed): 1 2

Enter fraction of files to be listed from each state (default: 1.0 (corresponding to all the␣
↪files)): 1.0

=== Output Preview ===

/path/to/ts/0001/run0001.dis
1 1
/path/to/ts/0001/run0002.dis
1 1
/path/to/ts/0002/run0001.dis
2 1
/path/to/ts/0002/run0002.dis
2 1
/path/to/ts/0003/run0001.dis
```

(continues on next page)

(continued from previous page)

```
3 0
/path/to/ts/0003/run0002.dis
3 0
/path/to/ts/0004/run0001.dis
4 0
/path/to/ts/0004/run0002.dis
4 0

Enter filename to save the file list: flist
Enter filename to save state_id and Rij_flag: funp
```

This produces a trajectory list `flist` and a file `funp` that describes which region is unperturbed in terms of dynamics in each time-series data.

Next, in order to generate `krhist` for each time-series dataset, the previously created `flist` and `funp` are split into per-line files using the following Python script:

```
$ $ANATRA_PATH/utility/iepdyn_util/split_cv.py --flist flist --funp funp
```

Executing this command generates the following directory structure:

```
0001/0001.flist
0001/0001.funp
0001/0002.flist
0001/0002.funp
0002/0001.flist
0002/0001.funp
0002/0002.flist
0002/0002.funp
0003/0001.flist
0003/0001.funp
0003/0002.flist
0003/0002.funp
0004/0001.flist
0004/0001.funp
0004/0002.flist
0004/0002.funp
```

The generated `XXXX.flist` and `XXXX.funp` files correspond to `flist_cv` and `f_unperturbed_id` for each time-series data. Therefore, `krhist` files can be generated sequentially using these inputs.

```
#!/usr/bin/env bash
```

(continues on next page)

(continued from previous page)

```
for i in {0001..0004};do
  for f in `ls $i/*.flist`;do
    num=`basename $f | sed -e "s/.flist//g"`
    flist=$i/${num}.flist
    funp=$i/${num}.funp

cat << EOF > $i/run${num}.inp
&input_param
 flist_cv = "$i/${num}.flist"
/
&output_param
 fhead = "$i/run${num}"
/
&option_param
 use_perturbed_traj   = .true.
 f_unperturbed_id     = "$i/${num}.funp"

 output_histogram     = .true.
 use_dissociate_state = .true.
 use_reflection_state = .false.
 use_product_state    = .false.
 calc_steady          = .false.
 calc_Pint            = .false.
 extrapolate          = .false.
 nstate               = 4
 ndim                 = 1
 nmol                 = 1
 dt                   = 2e-05
 t_sparse             = 2e-05
 t_range              = 1.0
 initial_state_ids    = 1 2
 dissociate_state_ids = 4
/
&state
 0.0     2.0      1.0
 2.0     3.5      1.0
 3.5     4.0      0.0
 4.0     1.0d10   0.0
/
EOF
    fanatra iepdyn $i/run${num}.inp >& $i/run${num}.out
```

(continues on next page)

(continued from previous page)

```
  done
done
```

Executing this produces the `krhist` files:

```
0001/run0001.krhist
0001/run0002.krhist
0002/run0001.krhist
0002/run0002.krhist
0003/run0001.krhist
0003/run0002.krhist
0004/run0001.krhist
0004/run0002.krhist
```

5.7.2.2 Calculation of Equilibrium Populations

As in the case without a bias potential, the initial population values for states 1 and 2 are computed by introducing reflecting boundary conditions.
The list of `krhist` files and related inputs are obtained again using `cvlist_generator.py`.
The following excerpt shows only the parts where user input is required:

```
(1) Enter the root directory to search: .
(2) Enter the target extension (e.g., txt): krhist
(3) Enter the maximum search directory depth: 2
(4) Enter directory level counted from the deepest directory for grouping (default: 1): 1
Did these files come from perturbed dynamics? (y/n): y
Enter state numbers used for Rij calculation (space-separated, empty input allowed): 1 2
Enter fraction of files to be listed from each state (default: 1.0 (corresponding to all the␣
↪files)): 1.0
Enter filename to save the file list: fkrhist.list
Enter filename to save state_id and Rij_flag: fkrhist.unp
```

```
#!/usr/bin/env bash

mkdir -p steady

# Create input file
#
cat << EOF > steady.inp
&input_param
  flist_cv = "fkrhist.list"
```

(continues on next page)

(continued from previous page)

```
/
&output_param
  fhead = "steady/out"
/
&option_param
  use_perturbed_traj = .false.
  f_unperturbed_id = "fkrhist.unp"

  input_type = "HISTOGRAM"
  output_histogram = .false.
  use_dissociate_state = .false.
  use_reflection_state = .true.
  use_product_state = .false.
  calc_steady = .true.
  calc_Pint = .false.
  extrapolate = .false.
  nstate = 4
  ndim = 1
  nmol = 1
  dt = 2e-05
  t_sparse = 2e-05
  t_range = 1.0
  initial_state_ids = 1 2
  reflection_state_ids = 4
/
&state
 0.0     2.0      1.0
 2.0     3.5      1.0
 3.5     4.0      0.0
 4.0     1.0d10   0.0
/
EOF

fanatra iepdyn steady.inp
```

Executing this produces the equilibrium population file `steady/out.steady`, which contains the equilibrium population values.

5.7.2.3 Calculation of $P_{\mathrm{B}}(t)$

The time evolution of $P_{\mathrm{B}}(t)$ is computed by setting `extrapolate = .true.`.

```
#!/usr/bin/env bash

mkdir -p extrapolate

# Create input file
#
cat << EOF > extrapolate.inp
&input_param
  flist_cv = "fkrhist.list"
/
&output_param
  fhead = "extrapolate/out"
/
&option_param
  use_perturbed_traj   = .true.
  f_unperturbed_id     = "fkrhist.unp"

  input_type           = "HISTOGRAM"
  output_histogram     = .false.
  use_dissociate_state = .true.
  use_reflection_state = .false.
  use_product_state    = .false.
  calc_steady          = .false.
  calc_Pint            = .false.
  extrapolate          = .true.
  nstate               = 4
  ndim                 = 1
  nmol                 = 1
  dt                   = 2e-05
  t_sparse             = 2e-05
  t_range              = 1.0
  t_extend             = 10.0
  dt_tcfout            = 2e-05
  initial_state_ids    = 1 2
  dissociate_state_ids = 4
/
&state
 0.0     2.0      0.9874940E+00
 2.0     3.5      0.1066799E-01
```

(continues on next page)

(continued from previous page)

```
 3.5     4.0      0.0
 4.0     1.0d10   0.0
/
EOF

fanatra iepdyn extrapolate.inp
```

5.7.2.4 Analytical Estimation of the Time Integral of $P_{\mathrm{B}}(t)$

This can be executed as follows:

```
#!/usr/bin/env bash

mkdir -p Pint

# Create input file
#
cat << EOF > Pint.inp
&input_param
  flist_cv = "fkrhist.list"
/
&output_param
  fhead = "Pint/out"
/
&option_param
  use_perturbed_traj   = .true.
  f_unperturbed_id     = "fkrhist.unp"

  input_type           = "HISTOGRAM"
  output_histogram     = .false.
  use_dissociate_state = .true.
  use_reflection_state = .false.
  use_product_state    = .false.
  calc_steady          = .false.
  calc_Pint            = .false.
  extrapolate          = .true.
  nstate               = 4
  ndim                 = 1
  nmol                 = 1
  dt                   = 2e-05
  t_sparse             = 2e-05
```

(continues on next page)

(continued from previous page)

```
  t_range              = 1.0
  t_extend             = 10.0
  dt_tcfout            = 2e-05
  initial_state_ids    = 1 2
  dissociate_state_ids = 4
/
&state
 0.0     2.0      0.9874940E+00
 2.0     3.5      0.1066799E-01
 3.5     4.0      0.0
 4.0     1.0d10   0.0
/
EOF

fanatra iepdyn Pint.inp
```

After execution, the file `Pint/out.int` is obtained.